

# Module 15 — Shortest Paths

---

**Sources:** CLRS 4e ch. 22 (Single-Source Shortest Paths) + ch. 23 (All-Pairs Shortest Paths)  
· Skiena 3e §8.3 (Shortest Paths — Dijkstra, Floyd–Warshall, transitive closure) + §8.4 (War Story: Dialing for Documents)

---

## Big Idea

**Problem.** Given a weighted, directed graph  $G = (V, E)$  with  $w : E \rightarrow \mathbb{R}$ , the **weight of a path**  $p = \langle v_0, v_1, \dots, v_k \rangle$  is  $w(p) = \sum_i w(v_{i-1}, v_i)$ , and the **shortest-path weight** is

$$\begin{aligned} \delta(u, v) &= \min \{ w(p) : u \rightsquigarrow v \} && \text{if any path exists} \\ &= \infty && \text{otherwise} \end{aligned}$$

A **shortest path** is any path achieving  $\delta(u, v)$ .

**Everything in this module is one operation applied in different orders.** That operation is **relaxation**:

```
RELAX(u, v, w)
  if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.π = u
```

→ **C++ implementation:** [A1 RELAX \(and INITIALIZE-SINGLE-SOURCE\)](#)

$v.d$  is an *upper-bound estimate* on  $\delta(s, v)$  that only ever decreases, and  $v.\pi$  is a predecessor pointer. Every single-source algorithm here initializes  $s.d = 0$ , everything else  $\infty$ , and then relaxes edges. **The only difference between them is the order in which edges are relaxed, and how many times.**

| ALGORITHM                 | RELAXATION ORDER                              | REQUIRES   | TIME              |
|---------------------------|-----------------------------------------------|------------|-------------------|
| <b>Bellman-Ford</b>       | every edge, $\lfloor  V /2 \rfloor - 1$ times | nothing    | $O( V E)$         |
| <b>DAG-shortest-paths</b> | every edge once, in topological order         | acyclic    | $\Theta( V  + E)$ |
| <b>Dijkstra</b>           | edges out of $u$ , in increasing $d[u]$ order | $w \geq 0$ | $O(E \lg  V )$    |

The reason the orderings differ in cost is a single structural fact — **Lemma 22.15 (path relaxation)**: if the edges of *some* shortest path to  $v$  get relaxed *in order*, then  $v.d = \delta(s, v)$  afterwards, no matter what else happened in between. So an algorithm only has to guarantee that ordering. Bellman-Ford brute-forces it (a shortest path has  $\leq |V| - 1$  edges, so  $\lfloor |V|/2 \rfloor - 1$  full passes must contain every shortest path as a subsequence). A DAG's topological order *is* that ordering, for free. Dijkstra earns it with a greedy argument that needs  $w \geq 0$ .

Then the all-pairs problem ( $\delta(u, v)$  for **every** pair) gets its own two answers: **Floyd-Warshall**  $\Theta(|V|^3)$ , a DP over "which vertices may be used as intermediates", and **Johnson**  $O(|V|^2 \lg |V| + |V|E)$ , which *reweights* the graph so Dijkstra becomes legal and then runs it  $|V|$  times.

**Remember months later:**  $\delta(u, v)$ , relaxation, and the six properties are the whole theory. Negative edges are the fault line: they break Dijkstra (not Bellman-Ford), and negative **cycles** break the problem itself ( $\delta = -\infty$ ). Sparse + non-negative  $\Rightarrow$  Dijkstra. Negative weights  $\Rightarrow$  Bellman-Ford. DAG  $\Rightarrow$  topological order, and remember it also gives you longest paths. All pairs, dense  $\Rightarrow$  Floyd-Warshall; all pairs, sparse  $\Rightarrow$  Johnson.

## What You Should Be Able To Do After This Chapter

- Define  $\delta(u, v)$ , state **Lemma 22.1** (subpaths of shortest paths are shortest paths), and explain why negative-weight cycles make  $\delta = -\infty$  and why shortest paths can be assumed **simple**.
- Write **RELAX** from memory and state the **six properties** (triangle inequality, upper-bound, no-path, convergence, path-relaxation, predecessor-subgraph) and which algorithm's correctness rests on which.
- Write **Bellman-Ford**, prove it (Lemma 22.2  $\rightarrow$  Theorem 22.4), and extend it to *report* a negative cycle and to mark  $-\infty$  vertices.
- Write **DAG-SHORTEST-PATHS**, and use it for **longest paths / critical paths (PERT)** — the one setting where longest paths are easy.
- Write **Dijkstra** with a binary heap and the lazy decrease-key idiom, give all three priority-queue complexities, and **prove Theorem 22.6**.

- Explain concretely — not just "it fails" — *how* Dijkstra fails on a negative edge, and why a `visited` set turns a slow algorithm into a wrong one.
  - Model a system of **difference constraints** as a shortest-path problem and solve it with Bellman-Ford.
  - Derive the **Floyd–Warshall** recurrence from the "intermediate vertices `{1..k}`" characterization, explain why `k` must be the **outermost** loop, and adapt it to **transitive closure**.
  - Explain **Johnson's reweighting**  $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ , prove it preserves shortest paths, and say why  $h(v) = \delta(s, v)$  from a super-source makes  $\hat{w} \geq 0$ .
  - Choose the right algorithm from `V`, `E`, sign of weights, and single-source vs all-pairs — in about five seconds, in an interview.
  - Recognize when a problem that *isn't about* distances is secretly a shortest-path problem (Skiena's war story; Viterbi).
- 

## Part 1 — Foundations (CLRS 22, preamble)

### Unified Understanding

Skiena opens with the practical framing: *"There are typically an enormous number of possible paths connecting two nodes in any given road or social network. The path that minimizes the sum of edge weights... is likely to be the most interesting, reflecting the fastest travel path or the closest kinship between the nodes."* And the key warning that motivates the whole chapter: **BFS is not enough**. In an unweighted graph, BFS's minimum-link path is the shortest path (M13, Lemma 20.6). In a weighted graph *"the shortest weighted path might require a large number of edges, just as the fastest route from home to office may involve complicated backroad shortcuts."*

CLRS supplies the formalism. Note the four problem variants and the fact that **they are all the same problem**:

| VARIANT                                              | REDUCTION                                                                       |
|------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>single-destination</b> $\delta(v, t)$ for all $v$ | reverse every edge, run single-source from $t$                                  |
| <b>single-pair</b> $\delta(u, v)$                    | no asymptotically better algorithm is known than solving single-source from $u$ |
| <b>all-pairs</b>                                     | ch. 23                                                                          |
| <b>single-source</b>                                 | the one we solve                                                                |

That single-pair line is worth internalizing: **there is no known way to compute one distance faster, in the worst case, than computing all  $|V|$  of them from that source.** (Practical speedups — A\*, bidirectional search — are in the Outside Context section, and they help enormously in practice without improving the worst case.)

## Lemma 22.1 — Optimal substructure

*Subpaths of shortest paths are shortest paths. If  $p = \langle v_0, \dots, v_k \rangle$  is a shortest path from  $v_0$  to  $v_k$ , then for any  $0 \leq i \leq j \leq k$ , the subpath  $p_{\{ij\}} = \langle v_i, \dots, v_j \rangle$  is a shortest path from  $v_i$  to  $v_j$ .*

**Proof skeleton (cut-and-paste — the standard DP/greedy move from M11/M12).**

Decompose  $p$  as  $v_0 \rightsquigarrow v_i \rightsquigarrow v_j \rightsquigarrow v_k$  with weight  $w(p) = w(p_{\{0i\}}) + w(p_{\{ij\}}) + w(p_{\{jk\}})$ . If some  $p'_{\{ij\}}$  were lighter, splicing it in gives a  $v_0 \rightsquigarrow v_k$  path of weight  $< w(p)$ , contradicting optimality of  $p$ . ■

This one lemma is the reason **every** algorithm in both chapters works. It is what makes  $v.\pi$  meaningful (a tree of shortest paths, not a pile of unrelated paths), it is the optimal-substructure step for Floyd–Warshall and for the matrix-multiplication DP, and it powers the triangle inequality.

**Skiena's version of the same insight**, stated as motivation for Dijkstra: "Suppose the shortest path from  $s$  to  $t$  in graph  $G$  passes through a particular intermediate vertex  $x$ . Clearly, the best  $s$ -to- $t$  path must contain the shortest path from  $s$  to  $x$  as its prefix, because if it doesn't we can improve the path by starting with the shorter  $s$ -to- $x$  prefix. Thus, we must compute the shortest path from  $s$  to  $x$  before we find the path from  $s$  to  $t$ ."

## Negative-weight edges and negative-weight cycles

These are **two different problems** and conflating them is the single most common confusion in this chapter.

- **Negative-weight edges** are fine.  $\delta$  is well-defined; Bellman-Ford handles them; only *Dijkstra* is broken by them.
- **Negative-weight cycles reachable from  $s$**  destroy the problem. Any path that can reach the cycle can go around it again for less, so no *shortest* path exists:  $\delta(s, v) = -\infty$  for every  $v$  reachable from such a cycle. CLRS defines  $\delta(s, v) = -\infty$  for exactly this case.

Two consequences people get wrong:

1. A negative-weight cycle that is **not reachable from  $s$**  is harmless —  $\delta$  is unaffected, and Bellman-Ford correctly returns `TRUE`. (Verified: a graph with  $s \rightarrow a$  in one component and a  $-5/-5$  two-cycle in another returns `ok = true`.)
2. Vertices reachable from  $s$  but **not** reachable from the negative cycle still have finite, correct  $\delta$ . Marking  $-\infty$  is a *reachability* computation on top of Bellman-Ford, not a global flag.

Skiena's version of why this matters, in the classic bank-lobby image:

*"During the execution we may encounter an edge with weight so negative that it changes the cheapest way to get from  $s$  to some other vertex already in the tree. Indeed, the most cost-effective way to get from your house to your next-door neighbor would be to repeatedly cycle through the lobby of any bank offering you enough free money to make the detour worthwhile. Unless that bank limits its reward to one per customer, you might so benefit by making an unlimited number of trips through the lobby that you would never actually reach your destination!"*

He then adds the practical note that keeps this in perspective: *"Fortunately, most applications don't have negative weights, making this discussion largely academic."* True for road networks. **Not** true for the settings where shortest paths get *used as a subroutine*: difference constraints (§22.4), Johnson's reweighting (§23.3), min-cost flow, currency arbitrage, and Viterbi decoding all produce negative weights on purpose.

## Can a shortest path contain a cycle?

**No** — you can always assume shortest paths are simple, with at most  $|V| - 1$  edges.

- **Negative-weight cycle:** then no shortest path exists at all (the case above).
- **Positive-weight cycle:** removing it strictly lowers the weight, so a path containing one is never shortest.
- **Zero-weight cycle:** removing it doesn't change the weight, so there is *also* a

simple shortest path. We take that one.

This is the fact that gives Bellman-Ford its  $|V| - 1$  and gives the matrix DP its  $L^{(n-1)}$ .

## Shortest-paths trees

The output is not a list of  $|V|$  separate paths; it is a **tree**. A shortest-paths tree rooted at  $s$  is a subgraph  $G' = (V', E')$  where:

1.  $V'$  is exactly the set of vertices reachable from  $s$ ,
2.  $G'$  is a rooted tree with root  $s$ ,
3. for every  $v \in V'$ , the unique simple  $s \rightarrow v$  path in  $G'$  is a shortest path in  $G$ .

Shortest-paths trees are **not unique** (ties in  $\delta$  give different trees of equal quality), and the tree produced depends on the relaxation order. **A shortest-paths tree is not the same object as a minimum spanning tree (M14)** — the MST minimizes total weight, the SPT minimizes each root-to-vertex distance. On the triangle  $s-a = 1$ ,  $s-b = 1$ ,  $a-b = 1.5$ , the MST is  $\{sa, ab\}$  (weight 2.5) but the SPT from  $s$  is  $\{sa, sb\}$  (weight 2). Different trees, different objectives.

## Relaxation — the one operation

`INITIALIZE-SINGLE-SOURCE` sets  $v.d = \infty$ ,  $v.\pi = \text{NIL}$ ,  $s.d = 0$ . `RELAX( $u, v, w$ )` asks "is going through  $u$  a better way to reach  $v$  than what I have?" and, if so, records it.

CLRS notes the terminology is historical and backwards-sounding: "the term is historical. The outcome of a relaxation step can be viewed as a relaxation of the constraint  $v.d \leq u.d + w(u, v)$ , which, by the triangle inequality, must be satisfied if  $u.d = \delta(s, u)$  and  $v.d = \delta(s, v)$ ." You are **tightening** an estimate by **relaxing** a violated constraint. Don't fight the name.

**One implementation trap that is not in the pseudocode:** if you represent  $\infty$  as a large finite number (`INT_MAX`, `LLONG_MAX`), then  $u.d + w(u, v)$  with  $u.d = \infty$  and  $w < 0$  is *less than*  $\infty$  and will relax a vertex you cannot even reach — and with `INT_MAX` it also overflows. Guard it: **never relax out of an unreachable vertex**. This is the `if (r.d[u] == INF) return false;` line below and it is the single most common source of wrong answers in negative-weight code.

The six properties (CLRS p. 611; proofs in §22.5)

Every correctness proof in ch. 22 is assembled from these six. Learn them as a unit.

| #     | PROPERTY                    | STATEMENT                                                                                                                                                                 | USED BY                                                       |
|-------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 22.10 | <b>Triangle inequality</b>  | $\delta(s, v) \leq \delta(s, u) + w(u, v)$ for all $(u, v) \in E$                                                                                                         | everything; and directly by Johnson to prove $\hat{w} \geq 0$ |
| 22.11 | <b>Upper-bound</b>          | $v.d \geq \delta(s, v)$ always; once $v.d = \delta(s, v)$ it never changes                                                                                                | everything                                                    |
| 22.12 | <b>No-path</b>              | if $v$ unreachable then $v.d = \delta(s, v) = \infty$ forever                                                                                                             | Bellman-Ford, Dijkstra                                        |
| 22.14 | <b>Convergence</b>          | if $s \rightsquigarrow u \rightarrow v$ is a shortest path and $u.d = \delta(s, u)$ before $\text{RELAX}(u, v, w)$ , then $v.d = \delta(s, v)$ after, forever             | Dijkstra, DAG                                                 |
| 22.15 | <b>Path-relaxation</b>      | if the edges of a shortest path $\langle v_0, \dots, v_k \rangle$ are relaxed <b>in order</b> , then $v_k.d = \delta(s, v_k)$ — regardless of any interleaved relaxations | <b>Bellman-Ford</b>                                           |
| 22.17 | <b>Predecessor-subgraph</b> | once $v.d = \delta(s, v)$ for all $v$ , $G_\pi$ is a shortest-paths tree                                                                                                  | all, for the <i>tree</i> half of the answer                   |

**Proof sketches, all short:**

- **Triangle inequality (22.10):** a shortest  $s \rightsquigarrow v$  path is no heavier than the particular path "shortest  $s \rightsquigarrow u$ , then edge  $(u, v)$ ".
- **Upper-bound (22.11):** induction on relaxation steps. Base:  $s.d = 0 \geq \delta(s, s)$  and  $\infty \geq \delta$  elsewhere. Step: if  $v.d$  changes it becomes  $u.d + w(u, v) \geq \delta(s, u) + w(u, v) \geq \delta(s, v)$  — inductive hypothesis then triangle inequality. And once  $v.d = \delta(s, v)$  it cannot decrease (it's a lower bound) and cannot increase (relaxation never increases  $d$ ).
- **No-path (22.12):** immediate corollary —  $\infty = \delta(s, v) \leq v.d$  forces  $v.d = \infty$ .
- **Lemma 22.13** (the helper): immediately after  $\text{RELAX}(u, v, w)$ ,  $v.d \leq u.d + w(u, v)$ . Either the relaxation fired (equality) or the condition already held.
- **Convergence (22.14):**  $v.d \leq u.d + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$  by 22.13, the hypothesis, and Lemma 22.1. Combined with the upper-bound property,  $v.d = \delta(s, v)$ .
- **Path-relaxation (22.15):** induction along the path, applying convergence at each edge.  $v_0.d = s.d = 0 = \delta(s, s)$  is the base.

- **Predecessor-subgraph (22.17):** first show  $G_\pi$  is always a rooted tree (Lemma 22.16 — a cycle in  $G_\pi$  would have to be a negative-weight cycle, by summing the  $v_i.d \geq v_{i-1}.d + w$  inequalities around it against the one strict inequality that created the cycle); then check the three defining properties of an SPT.

*Empirical check.* These are not decorative. Over 300 random graphs with negative edges (no negative cycles), relaxing a randomly chosen edge  $5/|E| + 10$  times and checking every invariant after **every single step**:  $d[v] \geq \delta(v)$  always held,  $d[v]$  never increased, unreachable vertices stayed  $\infty$ , converged vertices stayed converged, and  $d[v] \leq d[u] + w(u, v)$  held immediately after each relaxation. 843 separate path-relaxation instances (relaxing one shortest path's edges in order, with random unrelated relaxations interleaved) all converged immediately, exactly as Lemma 22.15 promises.

## C++ Implementation

```
const long long INF = LLONG_MAX / 4;    // "infinity" that survives
one addition

struct Edge { int u, v; long long w; };

struct Graph {                          // directed, weighted
    int n;
    vector<vector<pair<int, long long>>> adj;    // adj[u] = {(v,
w(u,v)), ...}
    vector<Edge> edges;                  // same edges,
flat (Bellman-Ford)

    explicit Graph(int n_) : n(n_), adj(n_) {}

    void addEdge(int u, int v, long long w) {
        adj[u].push_back({v, w});
        edges.push_back({u, v, w});
    }
};

struct SSSP {
    vector<long long> d;    // d[v] = shortest-path estimate
    vector<int> pi;        // pi[v] = predecessor, -1 = NIL
};
```



```

void initializeSingleSource(const Graph& g, int s, SSSP& r) {
    r.d.assign(g.n, INF);
    r.pi.assign(g.n, -1);
    r.d[s] = 0;
}

// The one operation every algorithm in this module is built from.
bool relax(int u, int v, long long w, SSSP& r) {
    if (r.d[u] == INF) return false;           // never relax out
    of "unreachable"
    if (r.d[v] > r.d[u] + w) {
        r.d[v] = r.d[u] + w;
        r.pi[v] = u;
        return true;
    }
    return false;
}

// Walk the predecessor chain backwards, then reverse. Empty  $\Rightarrow$  v
// unreachable.
vector<int> extractPath(const SSSP& r, int s, int v) {
    if (r.d[v] == INF) return {};
    vector<int> path;
    for (int x = v; x != -1; x = r.pi[x]) {
        path.push_back(x);
        if (x == s) break;
    }
    if (path.back() != s) return {};
    reverse(path.begin(), path.end());
    return path;
}

```

### Implementation notes.

- `INF = LLONG_MAX / 4` rather than `LLONG_MAX`: it survives `INF + w` for any sane `w` without overflowing, and it still compares as "unreachable" against any real distance. Never use `INT_MAX` with `+`.
- `relax` returns `bool` — this is what makes Bellman-Ford's early exit, Dijkstra's push, and the negative-cycle detector all one-liners.
- Storing edges **twice** (adjacency list + flat list) costs  $\Theta(E)$  memory and buys you both traversal orders. In a contest, keep only the one you need.

- `extractPath` walks  $\pi$  backwards. Skiena: "We follow the backward parent pointers from  $t$  until we hit start (or  $-1$  if no such path exists), exactly as was done in the BFS/DFS `find_path()` routine." Same routine as M13 — the tree structure is the same, only the tree differs.

## Part 2 — Bellman-Ford (CLRS 22.1)

### Unified Understanding

Bellman-Ford is the algorithm you get by refusing to be clever. **Relax every edge,  $|V| - 1$  times.** Then check once more: if anything still improves, there is a negative-weight cycle.

```

BELLMAN-FORD( $G, w, s$ )
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$            // the  $|V|$ -th pass is the
test                                     test
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE

```

→ C++ implementation: [A2 BELLMAN-FORD](#)

### Why it works

**The entire correctness argument is one application of the path-relaxation property.**

Take any vertex  $v$  reachable from  $s$ , and a shortest path  $p = \langle s = v_0, v_1, \dots, v_k = v \rangle$  to it. We argued above that  $p$  may be assumed **simple**, so  $k \leq |V| - 1$ . Now: pass 1 relaxes every edge, so in particular it relaxes  $(v_0, v_1)$ . Pass 2 relaxes  $(v_1, v_2)$ . In general, **pass  $i$  relaxes  $(v_{i-1}, v_i)$** . After  $k \leq |V| - 1$  passes, the edges of  $p$  have been relaxed *in order* (with lots of other relaxations interleaved, which Lemma 22.15 explicitly permits), so  $v.d = \delta(s, v)$ .

That's it. The algorithm is a brute-force way to guarantee the hypothesis of Lemma 22.15 for **every** shortest path simultaneously.

## Proof Skeleton

**Lemma 22.2.** If  $G$  has no negative-weight cycle reachable from  $s$ , then after  $|V| - 1$  passes,  $v.d = \delta(s, v)$  for all  $v$  reachable from  $s$ .

*Proof:* exactly the path-relaxation argument above. Unreachable vertices are handled by the no-path property.

**Corollary 22.3.**  $v.d = \delta(s, v)$  iff  $G_\pi$  contains a path  $s \rightsquigarrow v$ . (Combines Lemma 22.2 with the predecessor-subgraph property.)

**Theorem 22.4 (correctness of BELLMAN-FORD).**

- If there is no reachable negative cycle: by Lemma 22.2,  $v.d = \delta(s, v)$ , so for every edge the triangle inequality gives  $v.d = \delta(s, v) \leq \delta(s, u) + w(u, v) = u.d + w(u, v)$ . The check never fires; return **TRUE**. And  $G_\pi$  is a shortest-paths tree by 22.17.
- If there is a reachable negative cycle  $c = \langle v_0, \dots, v_k \rangle$ ,  $v_0 = v_k$ , with  $\sum w(v_{i-1}, v_i) < 0$ : suppose for contradiction the check does **not** fire, i.e.  $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$  for all  $i$ . Sum these  $k$  inequalities around the cycle:

$$\sum_i v_i.d \leq \sum_i v_{i-1}.d + \sum_i w(v_{i-1}, v_i)$$

Each vertex of  $c$  appears exactly once in each of the two  $d$ -sums (because  $v_0 = v_k$ ), so those sums are **equal and finite** — finite because the cycle is reachable, so every  $v_i.d < \infty$ . Cancel them:

$$0 \leq \sum_i w(v_{i-1}, v_i) = w(c) < 0$$

A contradiction. So the check fires and the algorithm returns **FALSE**. ■

**This "sum the inequalities around the cycle and cancel" move is worth memorizing verbatim.** It appears again, essentially unchanged, in Lemma 22.16 (the  $\pi$ -subgraph is acyclic), in Theorem 22.9 (difference constraints are infeasible iff the constraint graph has a negative cycle), and in Lemma 23.1 (Johnson's reweighting preserves negative cycles).

## Complexity

- **INITIALIZE-SINGLE-SOURCE:**  $\Theta(V)$ .
- **Main loop:**  $|V| - 1$  passes  $\times |E|$  relaxations  $\times \Theta(1)$  each =  $\Theta(VE)$ .
- **Check:**  $O(E)$ .
- **Total:**  $\Theta(VE)$  —  $\Theta(V^3)$  on a dense graph,  $\Theta(V^2)$  on a sparse one.
- **Space:**  $\Theta(V)$  auxiliary ( $d, \pi$ ), plus the graph. **No recursion**, so no stack term.

**Where the  $|V| - 1$  comes from:** it is exactly the maximum number of edges on a simple path. Not "a safety margin" — the tight bound.

*Empirical check.* On a random  $V = 200$ ,  $E = 995$  graph with weights in  $[1, 100]$ , Bellman-Ford **converged in 7 passes**, not 199 — random graphs have small diameter, so the early-exit test pays for itself many times over. On an adversarial instance (a 200-vertex path, with the edges added in reverse order so each pass advances the frontier by exactly one vertex), it needed **all 199 passes**. The bound is tight, and the constant factor in practice is a function of the graph's **shortest-path hop diameter**, not  $|V|$ .

**The early-exit optimization is free and always worth writing:** if a pass changes nothing, no later pass will either, so stop. It turns Bellman-Ford from  $\Theta(VE)$  into  $\Theta(hE)$  where  $h$  is the largest hop-count of any shortest path.

## C++ Implementation

```
struct BellmanFordResult {
    bool ok;           // false  $\Leftrightarrow$  a negative-weight cycle is
    reachable from s
    SSSP r;
};

BellmanFordResult bellmanFord(const Graph& g, int s) {
    BellmanFordResult res;
    initializeSingleSource(g, s, res.r);

    for (int i = 1; i < g.n; ++i) {                //  $|V| - 1$  passes
        bool changed = false;
        for (const Edge& e : g.edges)
            if (relax(e.u, e.v, e.w, res.r)) changed = true;
        if (!changed) break;                        // early exit:
    }
    already converged
}
```

```

        res.ok = true; // the |V|-th pass
is the test
        for (const Edge& e : g.edges)
            if (res.r.d[e.u] != INF && res.r.d[e.v] > res.r.d[e.u] +
e.w) { res.ok = false; break; }
        return res;
    }

    const long long NEG_INF = LLONG_MIN / 4;

// Exercise 22.1-4: after bellmanFord, set d[v] = -inf for every v
whose true
// shortest-path weight is -inf (reachable from a reachable
negative cycle).
void markNegativeInfinity(const Graph& g, SSSP& r) {
    vector<char> bad(g.n, 0);
    vector<int> stack;
    for (const Edge& e : g.edges) // still-relaxable
⇒ downstream of a neg. cycle
        if (r.d[e.u] != INF && r.d[e.v] > r.d[e.u] + e.w &&
!bad[e.v]) {
            bad[e.v] = 1; stack.push_back(e.v);
        }
    while (!stack.empty()) { // ...and
everything reachable from those
        int u = stack.back(); stack.pop_back();
        for (const auto& [v, w] : g.adj[u]) {
            (void)w;
            if (!bad[v]) { bad[v] = 1; stack.push_back(v); }
        }
    }
    for (int v = 0; v < g.n; ++v) if (bad[v]) r.d[v] = NEG_INF;
}

// Exercise 22.1-7: recover an actual negative-weight cycle
reachable from s.
// Returns v0 ... vk with v0 == vk, or {} if none exists.
vector<int> findNegativeCycle(const Graph& g, int s) {
    SSSP r;
    initializeSingleSource(g, s, r);
    int x = -1;

```

```

    for (int i = 0; i < g.n; ++i) {                                // |V| passes,
remembering the last change
        x = -1;
        for (const Edge& e : g.edges)
            if (relax(e.u, e.v, e.w, r)) x = e.v;
    }
    if (x == -1) return {};                                        // nothing relaxed
on pass |V|
    for (int i = 0; i < g.n; ++i) x = r.pi[x];                    // walk |V| steps
back: guaranteed onto the cycle
    vector<int> cycle;
    for (int v = x;; v = r.pi[v]) {
        cycle.push_back(v);
        if (v == x && cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return cycle;
}

```

**Why `findNegativeCycle` walks  $|V|$  steps back.** After  $|V|$  passes,  $x$  is a vertex that *still* relaxed, so  $x$  is reachable from a negative cycle via  $\pi$  pointers. The  $\pi$ -chain from  $x$  may enter the cycle after some prefix, but that prefix has fewer than  $|V|$  vertices, so  $|V|$  **steps backwards lands you inside the cycle for sure**. Then walk  $\pi$  until you see the same vertex again. This is a trick worth memorizing — it comes up constantly in competitive programming (currency arbitrage, "find a profitable loop").

*Empirical check.* Over 600 random 4–11 vertex graphs with weights in  $[-12, 8]$ , **415 had a reachable negative cycle**. In all 415, `findNegativeCycle` returned a genuine cycle (every consecutive pair an actual edge, first vertex = last vertex) whose total weight was **strictly negative**. And in all 415, `markNegativeInfinity` agreed exactly with an independent oracle:  $v$  is  $-\infty$  iff  $v$  is reachable from  $s$  **and** reachable from some vertex  $c$  with `Floyd-Warshall`  $d[c][c] < 0$  that is itself reachable from  $s$ .

## Practical variant: SPFA (queue-based Bellman-Ford)

Only relax out of vertices whose  $d$  actually changed. Keep them in a FIFO queue with an "in queue" flag. Same  $O(VE)$  worst case, dramatically faster typical case. Known as **SPFA** in competitive programming. Two caveats: (1) it is  $O(VE)$  in the worst case and adversarial test data targeting SPFA is common on judges; (2) with the **SLF** (small-label-first) and **LLL** heuristics it gets faster still, but never gains a better bound. Use it when you

need Bellman-Ford's generality and Bellman-Ford's constant hurts.

---

## Part 3 — Shortest Paths in a DAG (CLRS 22.2)

### Unified Understanding

If the graph is acyclic, **the topological order is the correct relaxation order, for free.**

```
DAG-SHORTEST-PATHS( $G, w, s$ )
1  topologically sort the vertices of  $G$ 
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )
3  for each vertex  $u$ , taken in topologically sorted order
4      for each vertex  $v \in G.\text{Adj}[u]$ 
5          RELAX( $u, v, w$ )
```

→ C++ implementation: [A3 DAG-SHORTEST-PATHS](#)

**One pass. Every edge relaxed exactly once. Negative weights are fine.**

### Why It Works — Theorem 22.5

**Proof skeleton.** Let  $v$  be reachable from  $s$  and  $p = \langle v_0 = s, \dots, v_k = v \rangle$  a shortest path. Because the graph is a DAG and  $p$  is a path,  $v_0, v_1, \dots, v_k$  appear **in that relative order** in *any* topological order (Lemma 20.11 / M13). The algorithm processes vertices in topological order and relaxes all of  $u$ 's out-edges when it reaches  $u$ , so it relaxes  $(v_0, v_1)$ , then  $(v_1, v_2)$ , ..., then  $(v_{k-1}, v_k)$  — **in order**. Path-relaxation property  $\Rightarrow v.d = \delta(s, v)$ . Vertices not reachable from  $s$  get  $\infty$  by the no-path property. ■

The topological order is the ordering that Bellman-Ford spends  $|V|-1$  passes brute-forcing. That's the entire speedup.

**Why negative edges are harmless here:** the argument never used  $w \geq 0$ . And a DAG has no cycles at all, so no negative cycles.

### Complexity

- Topological sort:  $\Theta(V + E)$  (DFS or Kahn — M13).

- Main loop: each vertex dequeued once, each edge relaxed once:  $\Theta(V + E)$ .
- **Total:**  $\Theta(V + E)$ . **Space**  $\Theta(V)$ .

This is the **fastest shortest-path algorithm in the module** — linear, and it tolerates negative weights. Whenever a problem's graph is acyclic, use it.

## The killer application: longest paths, PERT, and critical paths

Longest path is NP-hard in a general graph (M19). **In a DAG it is linear**, because you can just negate the weights:

$$\text{longest}(s, v) = -\delta_{\{-w\}}(s, v)$$

Negation is legal because there are no cycles to turn negative. (Contrast M14: negating weights works for MST too, but for shortest paths in a *general* graph it is catastrophic — negation manufactures negative cycles.)

CLRS's application is **PERT chart analysis**: vertices are jobs, an edge  $(u, v)$  means job  $u$  must precede job  $v$ , and the edge weight is  $u$ 's duration. The **critical path** — the longest path through the DAG — is the **minimum possible time to complete all jobs**, and any delay on it delays the whole project. Two equivalent implementations: negate the weights and run `DAG-SHORTEST-PATHS`, or replace `min` with `max` and  $\infty$  with  $-\infty$  in `RELAX` directly.

Skiena's version of the same idea, from the war story below: the Viterbi algorithm "is basically solving a shortest-path problem on a DAG."

## C++ Implementation

```
// Kahn topological order; empty result  $\Rightarrow$  the graph has a cycle.
vector<int> topoOrder(const Graph& g) {
    vector<int> indeg(g.n, 0), order;
    for (const Edge& e : g.edges) ++indeg[e.v];
    vector<int> q;
    for (int v = 0; v < g.n; ++v) if (indeg[v] == 0)
        q.push_back(v);
    while (!q.empty()) {
        int u = q.back(); q.pop_back();
        order.push_back(u);
        for (const auto& [v, w] : g.adj[u]) { (void)w; if (--
            indeg[v] == 0) q.push_back(v); }
    }
}
```



```

    return (int)order.size() == g.n ? order : vector<int>{};
}

SSSP dagShortestPaths(const Graph& g, int s) {
    SSSP r;
    initializeSingleSource(g, s, r);
    for (int u : topoOrder(g)) // one pass, in
topological order
        if (r.d[u] != INF)
            for (const auto& [v, w] : g.adj[u]) relax(u, v, w, r);
    return r;
}

// PERT / critical path: longest path in a DAG = shortest path with
negated weights.
SSSP dagLongestPaths(const Graph& g, int s) {
    Graph h(g.n);
    for (const Edge& e : g.edges) h.addEdge(e.u, e.v, -e.w);
    SSSP r = dagShortestPaths(h, s);
    for (long long& x : r.d) if (x != INF) x = -x;
    return r;
}

```

*Empirical check.* CLRS Figure 22.5 ( $r, s, t, x, y, z$  with source  $s$ ) reproduces exactly:  $d = [\infty, 0, 2, 6, 5, 3]$ , and Bellman-Ford on the same graph agrees. Over 400 random DAGs with weights in  $[-20, 20]$  (all of them containing negative edges), `dagShortestPaths` agreed with Bellman-Ford, Floyd-Warshall and Johnson from **every** source. `dagLongestPaths` agreed with a direct max-DP on 300 random DAGs, and on the 5-task example gives critical path  $0 \rightarrow 1 \rightarrow 3 \rightarrow 4$  of length 9.

## Part 4 — Dijkstra's Algorithm (CLRS 22.3, Skiena 8.3.1)

### Unified Understanding

**Requires**  $w(u, v) \geq 0$  for all edges. Maintain a set  $S$  of vertices whose shortest-path weights are **final**. Repeatedly extract the vertex  $u \notin S$  with the smallest  $u.d$ , add it to  $S$ , and relax all its out-edges.

```

DIJKSTRA( $G, w, s$ )
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = \emptyset$ 
4  for each vertex  $u \in G.V$ 
5      INSERT( $Q, u$ )
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8       $S = S \cup \{u\}$ 
9      for each vertex  $v \in G.\text{Adj}[u]$ 
10         if RELAX( $u, v, w$ )
11             DECREASE-KEY( $Q, v, v.d$ )

```

→ C++ implementation: [A4 DIJKSTRA](#)

**Skiena's framing is the one to keep**, because it makes the algorithm impossible to forget: "The basic idea is very similar to Prim's algorithm. In each iteration, we add exactly one vertex to the tree of vertices for which we know the shortest path from  $s$ . As in Prim's, we keep track of the best path seen to date for all vertices outside the tree, and insert them in order of increasing cost." And then the punchline — he presents Dijkstra as "an implementation of Dijkstra's algorithm based on changing exactly four lines from our Prim's implementation—one of which is simply the name of the function!"

*The only difference between Prim and Dijkstra:*

#### KEY $v$ IS RANKED BY

**Prim (M14)**       $w(u, v)$  — the weight of the connecting edge

**Dijkstra**       $d[u] + w(u, v)$  — the distance from  $s$  through  $u$

Skiena: "In the minimum spanning tree algorithm, we sought to minimize the weight of the next potential tree edge. In shortest path, we want to identify the closest outside vertex (in shortest-path distance) to  $s$ . This desirability is a function of both the new edge weight and the distance from  $s$  to the tree vertex it is adjacent to."

That one-line difference is also exactly why Prim tolerates negative weights and Dijkstra does not: Prim's keys don't accumulate.

**CLRS's framing** is the greedy one: Dijkstra "always chooses the 'lightest' or 'closest' vertex in  $V - S$  to add to set  $S$ " — an instance of the greedy strategy of M12. Note that this greedy choice is provably optimal (unlike, say, 0-1 knapsack), and the proof is Theorem 22.6.

## Why It Works — Theorem 22.6

**Loop invariant:** at the start of each iteration of the `while` loop,  $v.d = \delta(s, v)$  for all  $v \in S$ .

**Proof skeleton (the squeeze).** Suppose for contradiction  $u$  is the **first** vertex extracted with  $u.d \neq \delta(s, u)$ . Then  $u \neq s$  (since  $s.d = 0 = \delta(s, s)$  and  $s$  is extracted first), and  $u$  must be reachable (else  $u.d = \delta = \infty$  by the no-path property), so a shortest path  $p : s \rightsquigarrow u$  exists.

Just before  $u$  is extracted,  $s \in S$  and  $u \notin S$ . So walking  $p$  from  $s$ , there is a **first** vertex  $y \in p$  with  $y \notin S$ ; let  $x \in S$  be its predecessor on  $p$ . Decompose  $p$  as  $s \rightsquigarrow x \rightarrow y \rightsquigarrow u$ .

1.  $x.d = \delta(s, x)$  — because  $x \in S$  and  $u$  was the first bad extraction.
2. Edge  $(x, y)$  was relaxed when  $x$  was added to  $S$ , and  $x.d$  was already correct then, so by the **convergence property**  $y.d = \delta(s, y)$ .
3.  $y$  precedes  $u$  on a shortest path, so  $\delta(s, y) \leq \delta(s, u)$  — **this step needs non-negative weights**: the segment  $y \rightsquigarrow u$  has weight  $\geq 0$ .
4.  $u$  was chosen by `EXTRACT-MIN` over  $y$ , so  $u.d \leq y.d$ .

Chain them:

$$\begin{array}{ccccccc} \delta(s, y) & \leq & \delta(s, u) & \leq & u.d & \leq & y.d = \delta(s, y) \\ (3) & & (22.11) & & (4) & & (2) \end{array}$$

Everything is squeezed to equality, so  $u.d = \delta(s, u)$  — contradicting the assumption. ■

**Corollary 22.7:** on termination  $v.d = \delta(s, v)$  for all  $v$ , and  $G_\pi$  is a shortest-paths tree (by 22.17).

**Look at where  $w \geq 0$  entered: step 3, and only step 3.** That is the entire dependency. Everything else in the proof is sign-agnostic. Knowing *exactly which line* fails is what lets you answer "why doesn't Dijkstra work with negative edges?" in an interview with something better than "it just doesn't."

## Complexity — three implementations

The running time is  $\Theta(V)$  inserts +  $\Theta(V)$  extract-mins +  $\leq E$  decrease-keys, so:

| PRIORITY QUEUE      | INSERT     | EXTRACT-MIN    | DECREASE-KEY | TOTAL                           | BEST WHEN                        |
|---------------------|------------|----------------|--------------|---------------------------------|----------------------------------|
| array / linear scan | $O(1)$     | $O(V)$         | $O(1)$       | $\Theta(V^2 + E) = \Theta(V^2)$ | dense: $E = \Omega(V^2 / \lg V)$ |
| binary min-heap     | $O(\lg V)$ | $O(\lg V)$     | $O(\lg V)$   | $O((V + E) \lg V) = O(E \lg V)$ | sparse — the practical default   |
| Fibonacci heap      | $O(1)$ am. | $O(\lg V)$ am. | $O(1)$ am.   | $O(V \lg V + E)$                | theory; large constants          |

(The  $O(E \lg V)$  simplification assumes  $E \geq V$ , i.e. every vertex is reachable — otherwise write  $O((V+E) \lg V)$ .)

Where each term comes from — say this, don't just quote the bound:

- $V$  extract-mins, one per vertex, because each vertex enters  $S$  exactly once.
- $\leq E$  decrease-keys, because a decrease-key can only follow a successful relaxation, and each edge is relaxed once.
- So the total is  $V \cdot T_{\text{extract}} + E \cdot T_{\text{decrease}}$ . Read the table as that formula.

**CLRS's historical note is the memorable part:** Fibonacci heaps were *invented* to make this line  $O(1)$ . "Because the number of DECREASE-KEY operations is typically much greater than the number of EXTRACT-MIN operations, any method that reduces the amortized time of DECREASE-KEY without increasing the amortized time of EXTRACT-MIN yields an asymptotically faster implementation." That is the entire design brief of the Fibonacci heap (M09's potential method built it).

Skiena's implementation is the  $\Theta(V^2)$  array version, and he is explicit that this is a *feature*, not a shortcoming: "As implemented here, the complexity is  $O(n^2)$ , exactly the same running time as a proper version of Prim's algorithm. This is because, except for the extension condition, it is exactly the same algorithm as Prim's."

**Empirical check.** Heap pushes never exceeded  $E + 1$  on any tested graph:  $V=100$ ,  $E=198 \rightarrow 89$  pushes;  $V=400$ ,  $E=79794 \rightarrow 1882$  pushes. Note how far below the bound the count is on a random graph — most relaxations don't improve anything. Meanwhile the  $\Theta(V^2)$  version does a fixed  $V^2 = 160,000$  scan steps regardless. The crossover is real: at  $V=400$ ,  $E=3190$  the heap does 704 pushes against 160,000 scan steps, and at  $V=400$ ,  $E \approx V^2/2$  the heap's  $E \lg V$  finally loses to the array's  $V^2$ .

## The lazy decrease-key idiom (the one you should actually write)

`std::priority_queue` has no `decrease-key`. The standard workaround — the same one used for Prim in M14 — is to **push a new entry on every improvement and skip stale entries on pop**:

```
if (du > r.d[u]) continue;    // this entry is stale; a better one
                               is already in d[]
```

The queue can hold up to  $E$  entries instead of  $V$ , so the bound becomes  $O(E \lg E) = O(E \lg V)$  — **the same**, because  $\lg E \leq \lg V^2 = 2 \lg V$ . Memory goes from  $\Theta(V)$  to  $\Theta(E)$ . In exchange, you never write a heap. This is the right default.

## C++ Implementation

```
SSSP dijkstra(const Graph& g, int s) {
    SSSP r;
    initializeSingleSource(g, s, r);
    using P = pair<long long, int>;
    (d[v], v)
    priority_queue<P, vector<P>, greater<P>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > r.d[u]) continue; // stale
    entry: lazy decrease-key
        for (const auto& [v, w] : g.adj[u])
            if (relax(u, v, w, r)) pq.push({r.d[v], v});
    }
    return r;
}

//  $\Theta(V^2)$  Dijkstra: no heap, linear scan for the minimum. Wins on
// dense graphs.
SSSP dijkstraDense(const Graph& g, int s) {
    SSSP r;
    initializeSingleSource(g, s, r);
    vector<char> done(g.n, 0);
    for (int iter = 0; iter < g.n; ++iter) {
        int u = -1;
        long long best = INF;
```

```

        for (int i = 0; i < g.n; ++i)
            if (!done[i] && r.d[i] < best) { best = r.d[i]; u = i;
    }

    if (u == -1) break; // rest
is unreachable
    done[u] = 1;
    for (const auto& [v, w] : g.adj[u]) relax(u, v, w, r);
    }
    return r;
}

// Skiena's "design graphs, not algorithms": vertex costs become
edge weights.
SSSP vertexWeightedShortestPaths(int n, const vector<vector<int>>&
out,
                                const vector<long long>& cost, int
s) {
    Graph g(n);
    for (int u = 0; u < n; ++u)
        for (int v : out[u]) g.addEdge(u, v, cost[v]); // pay
for the vertex you enter
    SSSP r = dijkstra(g, s);
    if (r.d[s] != INF) r.d[s] = cost[s]; // plus
the source's own cost
    for (int v = 0; v < n; ++v) if (v != s && r.d[v] != INF) r.d[v]
+= cost[s];
    return r;
}

```

### Common bugs in Dijkstra, in order of how often they appear in real code:

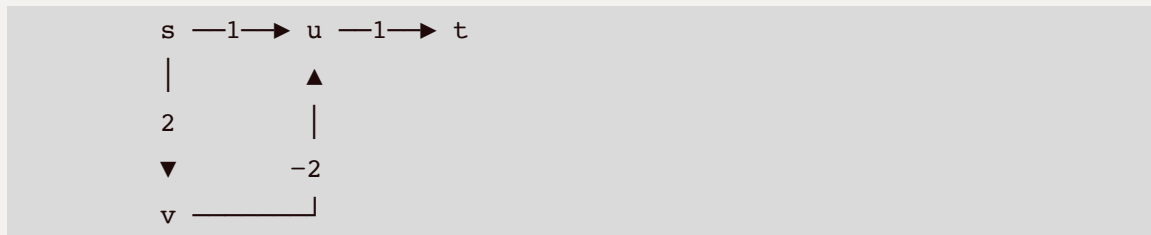
1. **Forgetting the stale-entry check** (`if (du > r.d[u]) continue;`). Still correct, but re-expands vertices and can blow up.
2. **Using `int` for distances**. `v * max_w` overflows `int` fast. Use `long long`.
3. **Comparator direction**. `std::priority_queue` is a **max**-heap by default. You need `greater<P>` (or push negated keys). Getting this backwards produces plausible-looking wrong answers.
4. **Pair ordering**. Push `{distance, vertex}`, not `{vertex, distance}` — the heap sorts on `.first`.
5. **Running it on a graph with negative edges** because "the weights are *mostly*

positive". See below.

## 6. Relaxing out of an unreachable vertex when $\infty$ is a finite sentinel.

### Why negative edges break it — concretely

Skiena's bank-lobby image (quoted in Part 1) explains negative *cycles*. But Dijkstra breaks on a single negative **edge**, with no cycle at all. Exercise 22.3-2 asks for exactly this, and the smallest instance is worth memorizing:



$\delta(s, u) = 0$  (via  $s \rightarrow v \rightarrow u$ , weight  $2 + (-2)$ ), so  $\delta(s, t) = 1$ . But Dijkstra extracts **u** **first**, with  $u.d = 1$ , because  $1 < 2$ . Step 3 of Theorem 22.6 has failed:  $y = v$  sits on the shortest path to **u** but  $\delta(s, v) = 2 > 0 = \delta(s, u)$ . Once **u** is in  $S$  and its out-edges have been relaxed with the wrong value,  $t.d = 2$  and never recovers.

*Empirical check.* On exactly this graph:  $\delta(s, u) = 0$  but **u** is finalized at **1**. `dijkstraDense` — which has an explicit `done[]` set, like the textbook — **returns** `d[t] = 2` when the truth is **1**. Bellman-Ford gets **1**.

**Here is the subtlety almost nobody knows, and it is a great interview follow-up.** The *lazy-heap* version above has **no visited set** — a stale-entry check is not the same thing. So when  $v.d$  improves after **v** was already popped, it simply gets pushed again and re-expanded. That makes it **correct on graphs with negative edges but no negative cycles** — and destroys the running-time guarantee, because the **v** extract-mins / **E** decrease-keys accounting no longer holds. It degenerates into a Bellman-Ford-flavoured algorithm (essentially SPFA with a heap).

*Empirical check.* On a chain of  $k$  "late-discovered discount" gadgets ( $v = 2k+1$ ,  $E = 3k$ , one route free, one route expensive-then-refunded so it pops last and re-triggers everything downstream), pops grow as  $(k+1)^2$ :

| $K$ | $V$ | $E$ | POPS |
|-----|-----|-----|------|
| 4   | 9   | 12  | 25   |
| 8   | 17  | 24  | 81   |
| 16  | 33  | 48  | 289  |
| 32  | 65  | 96  | 1089 |
| 64  | 129 | 192 | 4225 |

The answer was correct every time (checked against Bellman-Ford) but the pop count is  $\Theta(V^2)$  on a graph with  $\Theta(V)$  edges — nothing like  $O(E \lg V)$ . Instances forcing far worse than quadratic blow-up are known. **Summary: with a `visited` set, negative edges make Dijkstra wrong; without one, they make it slow. Neither is what you want — use Bellman-Ford, or reweight with Johnson.**

## Stop and Think: shortest path with node costs (Skiena)

**Problem.** Given a directed graph whose weights are on the **vertices** instead of the edges — the cost of a path from  $x$  to  $y$  is the sum of the weights of all vertices on it — find shortest paths.

Skiena gives two solutions and clearly prefers the second:

"A natural idea would be to adapt the algorithm we have for edge-weighted graphs (Dijkstra's) to the new vertex-weighted domain... However, my preferred approach would leave Dijkstra's algorithm intact and instead concentrate on constructing an edge-weighted graph on which Dijkstra's algorithm will give the desired answer. Set the weight of each directed edge  $(i, j)$  in the input graph to the cost of vertex  $j$ . Dijkstra's algorithm now does the job. **Try to design graphs, not algorithms.**"\*

That italicized sentence is one of the most valuable lines in Skiena's book, and it is the mental move that a huge fraction of hard interview graph problems reward. **Do not modify the algorithm. Modify the graph so a stock algorithm answers your question.** Some standard instances of the move:



| PROBLEM                                                 | GRAPH YOU BUILD                                                                            |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Costs on vertices                                       | $w(i, j) = \text{cost}(j)$ ; add $\text{cost}(s)$ at the end                               |
| Costs on both vertices and edges                        | $w(i, j) = \text{edge}(i, j) + \text{cost}(j)$                                             |
| At most $k$ "free" edges / refuels / discounts          | <b>layered graph</b> : $k+1$ copies of $v$ , free edges jump a layer                       |
| Alternating constraints (must alternate red/blue edges) | 2 copies of $v$ , one per state                                                            |
| Node capacity in flow (M16)                             | split $v$ into $v_{\text{in}} \rightarrow v_{\text{out}}$ with capacity on the middle edge |
| Shortest path with parity / mod- $m$ constraint         | $m$ copies of $v$                                                                          |
| Time-dependent edges (trains, schedules)                | vertices = (station, time)                                                                 |

Every one of these is Dijkstra or BFS on a **different graph**, not a different algorithm.

***Empirical check.** Over 300 random vertex-weighted graphs, the edge-weight transformation matched a direct vertex-cost relaxation computed independently, on every instance.*

## Part 5 — Difference Constraints and Shortest Paths (CLRS 22.4)

### Unified Understanding

This is where shortest paths stop being about distance. It is the best evidence in either book for "graph algorithms are a **modelling language**, not just a toolbox."

A **system of difference constraints** is a linear program  $Ax \leq b$  in which **every row has exactly one +1 and one -1**, i.e. every constraint has the form

$$x_j - x_i \leq b_k$$

Such systems show up whenever the unknowns are **times** and the constraints are "**at least / at most this much time must elapse between two events.**" CLRS's example: "If the manufacturer applies an adhesive that takes 2 hours to set at time  $x_1$  and has to wait until it sets to install a part at time  $x_2$ , then there is a constraint that  $x_2 \geq x_1 + 2$  or, equivalently, that  $x_1 - x_2 \leq -2$ ." Scheduling, timing analysis in circuits, temporal

databases and build systems are all shaped like this.

**Lemma 22.8 (the shift lemma).** If  $\mathbf{x}$  is a solution, so is  $\mathbf{x} + d \cdot \mathbf{1}$  for any constant  $d$ .

*Proof:*  $(x_j + d) - (x_i + d) = x_j - x_i$ . ■ So solutions come in translation classes — you can always normalize  $x_0 = 0$ .

Also note: if every  $b_i \geq 0$ , the system is trivially feasible (set all  $x_i$  equal). **The problem is interesting only when some  $b_i < 0$**  — which is exactly when negative edges appear.

## The constraint graph

View the  $m \times n$  matrix  $A$  as the **transpose of an incidence matrix**. Build  $G = (V, E)$ :

- $V = \{v_0, v_1, \dots, v_n\}$  — one vertex per unknown, **plus a super-source  $v_0$** ,
- for each constraint  $x_j - x_i \leq b_k$ , an edge  $(v_i, v_j)$  of weight  $b_k$ ,
- an edge  $(v_0, v_i)$  of weight  $0$  for every  $i$ .

The  $v_0$  edges exist for one reason: **to guarantee some vertex can reach all the others**, so a single Bellman-Ford run sees the whole graph (and so every negative cycle is reachable from the source and therefore detected).

## Theorem 22.9

*If  $G$  has **no negative-weight cycle**, then  $\mathbf{x} = (\delta(v_0, v_1), \dots, \delta(v_0, v_n))$  is a **feasible solution**. If  $G$  has a negative-weight cycle, the system has **no feasible solution**.*

**Proof skeleton — and it is beautifully short in both directions.**

*Feasibility.* For any edge  $(v_i, v_j)$  the **triangle inequality** gives  $\delta(v_0, v_j) \leq \delta(v_0, v_i) + w(v_i, v_j)$ , i.e.  $x_j - x_i \leq b_k$ . **That is literally the constraint.** The triangle inequality is the difference constraint. ■

*Infeasibility.* Let  $C = \langle v_1, \dots, v_k \rangle$ ,  $v_1 = v_k$ , be a negative cycle. ( $v_0$  cannot be on it — it has no entering edges.) The edges of  $C$  correspond to constraints  $x_2 - x_1 \leq w(v_1, v_2)$ , ...,  $x_k - x_{k-1} \leq w(v_{k-1}, v_k)$ . Sum them: on the left, each unknown is added once and subtracted once (since  $x_1 = x_k$ ), so the left side is  $0$ ; the right side is  $w(C)$ . So any solution would satisfy  $0 \leq w(C) < 0$ . Contradiction. ■

**Same move as Theorem 22.4.** Sum around the cycle, watch the  $x$ 's telescope to zero, contradict.

## Complexity

$m$  constraints on  $n$  unknowns  $\Rightarrow n + 1$  vertices and  $n + m$  edges  $\Rightarrow$  Bellman-Ford runs in  $O((n+1)(n+m)) = O(n^2 + nm)$ . Exercise 22.4-5 improves this to  $O(nm)$  even when  $m \ll n$  (the trick: the  $v_0$  edges never need re-relaxing, so initialize all  $d = 0$  and drop  $v_0$  entirely — Exercise 22.4-7).

Two exercises worth knowing as facts, because they turn a feasibility solver into an optimizer:

- **Ex 22.4-8:** Bellman-Ford's solution **maximizes**  $\sum x_i$  subject to  $Ax \leq b$  and  $x_i \leq 0$ .
- **Ex 22.4-9:** it also **minimizes**  $\max\{x_i\} - \min\{x_i\}$  — i.e. it produces the schedule with the **shortest makespan**. That is why this is genuinely useful for scheduling construction jobs, not just a cute reduction.

## C++ Implementation

```
struct Constraint { int i, j; long long b; }; //
means x[j] - x[i] <= b

bool solveDifferenceConstraints(int n, const vector<Constraint>&
cs, vector<long long>& x) {
    Graph g(n + 1);
    const int v0 = n; // the
extra super-source
    for (const Constraint& c : cs) g.addEdge(c.i, c.j, c.b);
    for (int i = 0; i < n; ++i) g.addEdge(v0, i, 0);
    BellmanFordResult res = bellmanFord(g, v0);
    if (!res.ok) return false; //
negative cycle  $\Rightarrow$  infeasible
    x.assign(n, 0);
    for (int i = 0; i < n; ++i) x[i] = res.r.d[i]; // x_i
=  $\delta(v_0, v_i)$ 
    return true;
}
```

**Empirical check.** CLRS's system (22.2)–(22.9) yields exactly the book's answer  $\mathbf{x} = (-5, -3, 0, -1, -4)$ , every constraint verified, and shifting by  $-100, +7, +1000$  stays feasible (Lemma 22.8). A two-constraint cycle  $x_1 - x_0 \leq -1, x_0 - x_1 \leq -1$  is correctly rejected. Over **400 random systems**: 283 reported feasible — every returned  $\mathbf{x}$  satisfies every constraint; 117 reported infeasible — **an exhaustive search over the box  $[-6, 6]^3$  with  $x_0 = 0$  fixed (legal by Lemma 22.8) found no solution in any of them.**

**Recognition pattern.** If a problem's constraints are all of the form " $b$  happens at least/at most  $k$  after  $a$ ", or "the difference between these two quantities is bounded", it is a difference-constraint system: **build the constraint graph and run Bellman-Ford.** Infeasibility = negative cycle. This also covers equality constraints  $x_i = x_j + b$  (encode as  $\leq b$  and  $\geq b$ , i.e. two edges — Ex 22.4-6) and single-variable bounds  $x_i \leq b$  (an edge from  $v_0$  — Ex 22.4-10).

## Part 6 — All-Pairs Shortest Paths: The Setup (CLRS 23, preamble)

### Unified Understanding

Now compute  $\delta(u, v)$  for **every** ordered pair. Output is an  $n \times n$  matrix  $D$  plus (optionally) an  $n \times n$  predecessor matrix  $\Pi$ .

**The naive answers, which are also sometimes the right answers:**

| APPROACH                             | TIME                                   | USE WHEN                                                   |
|--------------------------------------|----------------------------------------|------------------------------------------------------------|
| $V \times$ Dijkstra (array queue)    | $O(V^3 + VE) = O(V^3)$                 | dense, $w \geq 0$                                          |
| $V \times$ Dijkstra (binary heap)    | $O(V(V+E) \lg V) = O(VE \lg V)$ sparse | <b>sparse, <math>w \geq 0</math></b>                       |
| $V \times$ Dijkstra (Fibonacci heap) | $O(V^2 \lg V + VE)$                    | sparse, $w \geq 0$ , theory                                |
| $V \times$ Bellman-Ford              | $O(V^2E) = O(V^4)$ dense               | negative weights — <b>and this is what we want to beat</b> |
| Floyd–Warshall                       | $\Theta(V^3)$                          | dense, negative weights OK                                 |
| Johnson                              | $O(V^2 \lg V + VE)$                    | <b>sparse, negative weights</b>                            |

**Representation switch.** Chapter 22 used adjacency lists. Chapter 23 uses an **adjacency matrix**, and Skiena explains why this is not extravagant: "*Floyd's algorithm is best employed on an adjacency matrix data structure, which is no extravagance since we must store all  $n^2$  pairwise distances anyway.*" The output is  $\Theta(V^2)$  no matter what, so the  $\Theta(V^2)$  input representation is free. Johnson is the exception — it uses adjacency lists, because it is the *sparse-graph* algorithm.

The input convention (23.1):

|              |                                     |
|--------------|-------------------------------------|
| $w_{ij} = 0$ | if $i = j$                          |
| $= w(i, j)$  | if $i \neq j$ and $(i, j) \in E$    |
| $= \infty$   | if $i \neq j$ and $(i, j) \notin E$ |

**Skiena's warning on this is the single most practical sentence in his shortest-paths section:**

*"The critical issue in an adjacency matrix implementation is how we denote the edges absent from the graph. A common convention for unweighted graphs denotes graph edges by 1 and non-edges by 0. This gives exactly the wrong interpretation if the numbers denote edge weights, because the non-edges get interpreted as a free ride between vertices. Instead, we should initialize each non-edge to MAXINT."*

A 0 in a weight matrix means "free edge," not "no edge." This bug produces answers that look reasonable and are entirely wrong.

**The honest motivating application.** CLRS punctures the standard one: a road atlas distance table sounds like all-pairs shortest paths, but "*a road map modeled as a graph has one vertex for every road intersection... the United States has approximately 300,000 signal-controlled intersections*" while the table has 100 cities. The **legitimate** application is the **diameter** of a network — the longest of all shortest paths, i.e. worst-case message transit time. Skiena adds the **graph center** — "*the one that minimizes the longest or average distance to all the other nodes. This might be the best place to start a new business.*"

The matrix-multiplication view (CLRS 23.1) — worth knowing for the idea, not the algorithm

Define  $l_{ij}^r =$  minimum weight of any path from  $i$  to  $j$  **using at most  $r$  edges**. Then  $l_{ij}^0 = 0$  if  $i = j$ , else  $\infty$ , and

$$l_{ij}^r = \min \{ l_{ik}^{r-1} + w_{kj} : 1 \leq k \leq n \}$$

(23.3)

(The "or don't extend at all" case  $l_{ij}^{(r-1)}$  is absorbed because  $w_{jj} = 0$ .)

Since shortest paths are simple and have  $\leq n - 1$  edges,  $\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = \dots$  (23.4).

**Now the observation the section exists for.** Compare (23.3) with ordinary matrix multiplication  $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$ :

| MATRIX MULTIPLY           | EXTEND-SHORTEST-PATHS        |
|---------------------------|------------------------------|
| $\Sigma$                  | $\min$                       |
| $\cdot$                   | $+$                          |
| identity for $\Sigma = 0$ | identity for $\min = \infty$ |
| identity for $\cdot = 1$  | identity for $+$ = 0         |

**EXTEND-SHORTEST-PATHS** is matrix multiplication over the  $(\min, +)$  semiring — the **tropical semiring**. So  $L^{(r)} = L^{(r-1)} \odot W = W^r$ , and the answer is  $W^{(n-1)}$ .

- Naive:  $n - 1$  products  $\times \Theta(n^3)$  each =  $\Theta(n^4)$  (SLOW-APSP).
- **Repeated squaring** (the same trick as fast modular exponentiation): the  $\odot$  product is associative, and  $L^{(r)} = L^{(n-1)}$  for all  $r \geq n - 1$ , so compute  $W^1, W^2, W^4, \dots, W^{(2^{\lceil \lg(n-1) \rceil})}$  in  $\lceil \lg(n-1) \rceil$  products =  $\Theta(n^3 \lg n)$  (FASTER-APSP).

**Why learn this if Floyd–Warshall beats it?** Three reasons:

1. **The semiring swap is a reusable technique.** Change  $(\min, +)$  to  $(\vee, \wedge)$  and you get transitive closure. Change it to  $(+, \times)$  and the same code **counts paths**. Change it to  $(\max, \min)$  and it computes **bottleneck** (widest-path) capacities. Same code, different algebra. (M11 used exactly this move to turn CYK parsing into minimum-edit CYK.)
2. **Repeated squaring generalizes.** "Shortest path using exactly / at most  $k$  edges" is  $W^k$  and computable in  $\Theta(n^3 \lg k)$  — a genuinely useful competitive-programming primitive that Floyd–Warshall cannot do.
3. It is the bridge to sub-cubic results: fast matrix multiplication does **not** directly apply (the tropical semiring lacks subtraction, so Strassen-style algorithms don't work), which is why  $\Theta(v^3)$  has been so hard to beat. The best known is  $O(v^3 (\lg v / \lg v)^{1/3})$  (Fredman) and later  $v^3 / 2^{\Omega(\sqrt{\lg v})}$  bounds — polylog improvements, not polynomial ones.

## Part 7 — The Floyd–Warshall Algorithm (CLRS 23.2, Skiena 8.3.2)

### Unified Understanding

**A different characterization of a shortest path — and a much better one.** Instead of counting edges, restrict which vertices may be used as **intermediates**.

An **intermediate vertex** of a path  $p = \langle v_1, v_2, \dots, v_l \rangle$  is any vertex other than  $v_1$  and  $v_l$ .

Number the vertices  $1..n$  — Skiena stresses the numbering carries no meaning: "We use these numbers not to label the vertices, but to order them."

Let  $d_{ij}^{(k)}$  = weight of a shortest  $i \rightsquigarrow j$  path **all of whose intermediate vertices lie in  $\{1, 2, \dots, k\}$** .

- $k = 0$ : no intermediates allowed at all, so  $d_{ij}^{(0)} = w_{ij}$  — **the adjacency matrix itself**.
- $k = n$ : everything allowed, so  $d_{ij}^{(n)} = \delta(i, j)$  — **the answer**.

### The recurrence and why it's correct

For each pair  $(i, j)$ , consider a shortest path  $p$  with intermediates in  $\{1..k\}$ . **Either  $k$  is on  $p$ , or it isn't.**

- **$k$  not on  $p$** : every intermediate of  $p$  is in  $\{1..k-1\}$ , so  $d_{ij}^{(k)} = d_{ij}^{(k-1)}$ .
- **$k$  on  $p$** : decompose  $p$  as  $i \rightsquigarrow k \rightsquigarrow j$ . By **Lemma 22.1**,  $p_1 = i \rightsquigarrow k$  and  $p_2 = k \rightsquigarrow j$  are themselves shortest paths. And here is the crucial detail:  **$k$  is not an intermediate vertex of  $p_1$  or of  $p_2$**  — it is an *endpoint* of each. (Shortest paths are simple, so  $k$  appears exactly once on  $p$ .) So both subpaths have all intermediates in  $\{1..k-1\}$ , giving  $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ .

$$\begin{aligned} d_{ij}^{(0)} &= w_{ij} \\ d_{ij}^{(k)} &= \min ( d_{ij}^{(k-1)} , \quad d_{ik}^{(k-1)} + d_{kj}^{(k-1)} ) \quad \text{for} \\ k &\geq 1 \end{aligned} \quad (23.6)$$

Skiena writes it identically and, being honest, flags the subtlety: "*The correctness of this is somewhat subtle, and I encourage you to convince yourself of it. Indeed, it is a great example of dynamic programming.*" **The subtle step is exactly the " $k$  is an endpoint, not an intermediate, of each half" observation.** If you can't produce that sentence, you don't yet understand Floyd–Warshall.

```
FLOYD-WARSHALL(W, n)
1  D^(0) = W
2  for k = 1 to n
3      let D^(k) be a new n × n matrix
4      for i = 1 to n
5          for j = 1 to n
6              d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
7  return D^(n)
```

→ C++ implementation: [A5 FLOYD-WARSHALL](#)

## The one thing to never get wrong: loop order

**$k$  must be the OUTERMOST loop.** This is the most common Floyd–Warshall bug in existence, and it is silent — the code compiles, runs in the same time, and returns wrong answers on some graphs.

The reason is the DP structure, not a coincidence: stage  $k$  depends on *all* of stage  $k-1$ . Writing `for i / for j / for k` computes, for each pair, some ad-hoc mixture of stages and never establishes the invariant "all paths through  $\{1..k\}$  have been found."

**Mnemonic:**  $k$  is the DP dimension;  $i$  and  $j$  are just the table you fill in at each stage.

**Exercise 23.2-4 — you can drop the superscripts.** The in-place version, which is what everyone writes (and what Skiena prints), is correct: during iteration  $k$ , the entries  $d[i][k]$  and  $d[k][j]$  never change, because  $d_{ik}^{(k)} = \min(d_{ik}^{(k-1)}, d_{ik}^{(k-1)} + d_{kk}^{(k-1)})$  and  $d_{kk}^{(k-1)} = 0$  (no negative cycles). So reading a partially-updated row or column is harmless. **Space drops from  $\Theta(n^3)$  to  $\Theta(n^2)$ .**

## Complexity

- Three nested loops,  $\Theta(1)$  inside:  $\Theta(n^3) = \Theta(v^3)$ , with no dependence on  $E$ .
- **Space  $\Theta(v^2)$**  (in-place), or  $\Theta(v^2)$  for  $D$  plus  $\Theta(v^2)$  for  $\Pi$ .
- **$\Theta$ , not  $o$**  — there is no early exit, no data-dependence, no best case. It always



does exactly  $n^3$  iterations.

### Why it beats $v \times$ Dijkstra in practice despite matching or losing asymptotically.

Skiena is blunt and correct: "The Floyd–Warshall all-pairs shortest-path algorithm runs in  $O(n^3)$  time, which is asymptotically no better than  $n$  calls to Dijkstra's algorithm. However, the loops are so tight and the program so short that it runs better in practice. It is notable as one of the rare graph algorithms that work better on adjacency matrices than adjacency lists." Three flat arrays, unit-stride inner loop, perfect branch prediction, no heap, no pointer chasing, trivially vectorizable and cache-blockable. **Constant factors matter, and this is the canonical example.**

*Empirical check.* CLRS Figure 23.1/23.4's five-vertex matrix reproduces exactly, and the in-place variant returns the identical matrix. At  $v = 400$ : on a **sparse** graph ( $E = 1596$ ) Floyd–Warshall took **37.5 ms** and Johnson **19.6 ms** — Johnson wins. On a **dense** graph ( $E = 95,767$ ) Floyd–Warshall took **59.9 ms** and Johnson **135.8 ms** — Floyd–Warshall wins by more than  $2\times$ . Note also that Floyd–Warshall's time barely moved between the two ( $37\text{ ms} \rightarrow 60\text{ ms}$ ) because it is  $\Theta(v^3)$  regardless of  $E$ , while Johnson's grew  $7\times$ .

## Constructing shortest paths

Two options.

**(a) Compute  $\Pi$  alongside  $D$ .**  $\pi_{ij}^k(k)$  = predecessor of  $j$  on a shortest  $i \rightsquigarrow j$  path with intermediates in  $\{1..k\}$ .

$$\begin{aligned} \pi_{ij}^0(0) &= \text{NIL} && \text{if } i = j \text{ or } w_{ij} = \infty \\ &= i && \text{otherwise} \\ (23.7) \end{aligned}$$
$$\begin{aligned} \pi_{ij}^k(k) &= \pi_{ij}^{k-1}(k-1) && \text{if } d_{ij}^{k-1}(k-1) \leq d_{ik}^{k-1}(k-1) + \\ & && d_{kj}^{k-1}(k-1) \\ &= \pi_{kj}^{k-1}(k-1) && \text{otherwise} \\ (23.8) \end{aligned}$$

**Read (23.8) carefully — it is  $\pi_{kj}$ , not  $\pi_{ik}$  and not  $k$ .** When routing through  $k$ , the last hop into  $j$  is whatever the last hop into  $j$  was on the  $k \rightsquigarrow j$  path. Row  $i$  of  $\Pi$  is then a shortest-paths **tree** rooted at  $i$ , and `PRINT-ALL-PAIRS-SHORTEST-PATH` walks it exactly like Chapter 22's `PRINT-PATH`.

**(b) Skiena's variant: store the *middle* vertex.** "These paths can be recovered if we retain a parent matrix  $P$  containing our choice of the last intermediate vertex used for each vertex pair  $(x, y)$ . Say this value is  $k$ . The shortest path from  $x$  to  $y$  is the concatenation of the shortest path from  $x$  to  $k$  with the shortest path from  $k$  to  $y$ , which can be reconstructed recursively given the matrix  $P$ ." Store  $\text{mid}[i][j] = k$  whenever the  $k$ -route wins; reconstruct by recursing on  $(i, k)$  and  $(k, j)$ . Equivalent; some find it more natural.

And Skiena's practical note, which is why he doesn't bother: "most all-pairs applications only need the resulting distance matrix. These are the jobs that Floyd's algorithm was designed for."

## Transitive closure (CLRS 23.2, Skiena 8.3.3) — the semiring swap

The **transitive closure**  $G^* = (V, E^*)$  has  $(i, j) \in E^*$  iff  $G$  has a path  $i \rightsquigarrow j$ .

**Method 1 (Skiena's).** Run Floyd–Warshall with all edge weights 1. "If the shortest path from  $i$  to  $j$  remains MAXINT after running Floyd's algorithm, you can be sure that no directed path exists from  $i$  to  $j$ . Any vertex pair of weight less than MAXINT must be reachable."  $\Theta(V^3)$ , works, wastes memory on long longs.

**Method 2 (CLRS's, better).** Substitute the boolean semiring:  $\min \rightarrow \vee$ ,  $+$   $\rightarrow \wedge$ , weights  $\rightarrow \{0, 1\}$ .

$$\begin{aligned} t_{ij}^{(0)} &= 1 \text{ if } i = j \text{ or } (i, j) \in E, \text{ else } 0 \\ t_{ij}^{(k)} &= t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}) \\ (23.9) \end{aligned}$$

Still  $\Theta(V^3)$  time but with **boolean** values, so it uses  $\Theta(V^2)$  bits instead of words — and it **bitset-vectorizes**: pack row  $j$  into a `std::bitset<N>` and the inner loop becomes `if (t[i][k]) t[i] |= t[k];`, giving a  $\Theta(V^3/w)$  algorithm with  $w = 64$ . That's a 64× constant-factor win and it is the standard trick.

**Skiena's blackmail graph** is the memorable framing. "Consider the blackmail graph, where there is a directed edge  $(i, j)$  if person  $i$  has sensitive-enough private information on person  $j$  so that  $i$  can get  $j$  to do whatever they want. You wish to hire one of these  $n$  people to be your personal representative. Who has the most power in terms of blackmail potential?" The naive answer is the vertex of highest out-degree; the right answer is the vertex that **reaches** the most others. "Steve might only be able to blackmail Miguel directly, but if Miguel can blackmail everyone else then Steve is the person you want to hire."

***Empirical check.** Over 300 random graphs, the  $(\vee, \wedge)$  closure agreed exactly with BFS-from-every-vertex **and** with the  $d < \text{INF}$  characterization. On the blackmail graph (Steve  $\rightarrow$  Miguel  $\rightarrow$  everyone; a decoy vertex with out-degree 2):*

*outdeg(Steve) = 1 but reach(Steve) = 6, versus reach(decoy) = 2 — **out-degree is the wrong metric, closure is the right one.***

## C++ Implementation

```
struct APSP {
    vector<vector<long long>> d;
    vector<vector<int>> pi; // -1 =
NIL
};

APSP floydWarshall(const vector<vector<long long>>& W) {
    int n = (int)W.size();
    APSP a;
    a.d = W;
    a.pi.assign(n, vector<int>(n, -1));
    for (int i = 0; i < n; ++i) //
(23.7)
        for (int j = 0; j < n; ++j)
            if (i != j && W[i][j] < INF) a.pi[i][j] = i;

    for (int k = 0; k < n; ++k) //
(23.6) — k outermost!
        for (int i = 0; i < n; ++i) {
            if (a.d[i][k] == INF) continue; //
prune: nothing through k
            for (int j = 0; j < n; ++j) {
                if (a.d[k][j] == INF) continue;
                long long through = a.d[i][k] + a.d[k][j];
                if (through < a.d[i][j]) {
                    a.d[i][j] = through;
                    a.pi[i][j] = a.pi[k][j]; //
(23.8)
                }
            }
        }
    return a;
}
```

```

// Exercise 23.2-4: the superscripts are unnecessary – update in
place.
vector<vector<long long>> floydWarshallInPlace(vector<vector<long
long>> d) {
    int n = (int)d.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (d[i][k] != INF && d[k][j] != INF && d[i][k] +
d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
    return d;
}

void printAllPairsPath(const APSP& a, int i, int j, vector<int>&
out) {
    if (i == j) out.push_back(i);
    else if (a.pi[i][j] == -1) out.clear(); // no
path
    else { printAllPairsPath(a, i, a.pi[i][j], out);
out.push_back(j); }
}

// (23.9): the same recurrence over (v, ^) instead of (min, +).
vector<vector<char>> transitiveClosure(const vector<vector<char>>&
adj) {
    int n = (int)adj.size();
    vector<vector<char>> t = adj;
    for (int i = 0; i < n; ++i) t[i][i] = 1;
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            if (t[i][k])
                for (int j = 0; j < n; ++j)
                    if (t[k][j]) t[i][j] = 1;
    return t;
}

```

The `if (a.d[i][k] == INF) continue;` line is the single most valuable micro-optimization in Floyd–Warshall. It hoists a test out of the inner loop and, on sparse graphs, skips the vast majority of the  $n^3$  iterations. It also removes the overflow risk from `INF + INF`.

### Extra facts worth carrying:

- $d[i][i] < 0$  after Floyd–Warshall  $\Leftrightarrow$  vertex  $i$  lies on a negative-weight cycle. This is the cheapest global negative-cycle detector there is, and it also tells you *which* vertices are involved. (Used as the independent oracle in the Bellman-Ford  $-\infty$  test above.)
  - **Graph diameter** =  $\max_{\{i,j\}} d[i][j]$  over finite entries. **Graph center** =  $\operatorname{argmin}_i \max_j d[i][j]$ .
  - Replace  $(\min, +)$  with  $(\max, \min)$  and you get the **widest path / bottleneck** matrix — the maximum over paths of the minimum edge capacity. (Connects to M14's minimum-bottleneck spanning tree.)
  - Replace  $(\min, +)$  with  $(+, \times)$  on a DAG's adjacency matrix and you **count paths**.
- 

## Part 8 — Johnson's Algorithm for Sparse Graphs (CLRS 23.3)

### Unified Understanding

**The problem:** on a sparse graph,  $V \times V$  Dijkstra ( $O(VE \lg V)$ ) crushes Floyd–Warshall ( $\Theta(V^3)$ ) — but Dijkstra needs  $w \geq 0$ .

**The wrong fix**, which is Exercise 23.3-4 and which everyone proposes first: let  $w^* = \min\{w(u,v)\}$  and set  $\hat{w}(u,v) = w(u,v) - w^*$ . **This does not work**, and knowing *why* is the point of the exercise: subtracting a constant per **edge** penalizes paths with more edges. A 2-edge path loses  $2w^*$  and a 5-edge path loses  $5w^*$ , so the ranking changes. **Any reweighting must be path-length-independent.**

**The right fix — telescoping.** Assign each **vertex** a value  $h(v)$  and set

$$\hat{w}(u,v) = w(u,v) + h(u) - h(v)$$

Now sum along a path  $p = \langle v_0, \dots, v_k \rangle$ :

$$\begin{aligned} \hat{w}(p) &= \sum_i [w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)] \\ &= w(p) + h(v_0) - h(v_k) \quad \leftarrow \text{every interior } h \text{ cancels} \end{aligned}$$

**The correction depends only on the endpoints, not on the path.** So for a fixed  $(u, v)$ , every  $u \rightsquigarrow v$  path shifts by the same amount  $h(u) - h(v)$ , and the ranking is preserved. That is Lemma 23.1.

## Lemma 23.1

With  $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ :

1.  $p$  is a shortest  $u \rightsquigarrow v$  path under  $w$  iff it is one under  $\hat{w}$ , and  $\hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$ .
2.  $G$  has a negative-weight cycle under  $w$  iff it does under  $\hat{w}$ .

*Proof of (1):* the telescoping computation above, applied to every path. *Proof of (2):* for a cycle  $c$  with  $v_0 = v_k$ ,  $\hat{w}(c) = w(c) + h(v_0) - h(v_k) = w(c)$ . **Cycle weights are completely unchanged.** ■

## Choosing $h$ — and this is the elegant part

We need  $\hat{w}(u, v) \geq 0$ , i.e.  $w(u, v) + h(u) - h(v) \geq 0$ , i.e.

$$h(v) - h(u) \leq w(u, v) \quad \text{for every edge } (u, v)$$

**That is a system of difference constraints** (§22.4) — the same object as Part 5. And Theorem 22.9 already told us a feasible solution: shortest-path weights from a super-source.

So: build  $G' = G$  plus a new vertex  $s$  with a weight-0 edge to every vertex, run **Bellman-Ford** from  $s$ , and set  $h(v) = \delta(s, v)$ . Then the triangle inequality gives, for every edge  $(u, v)$ ,

$$\delta(s, v) \leq \delta(s, u) + w(u, v) \iff h(v) \leq h(u) + w(u, v) \iff \hat{w}(u, v) \geq 0$$

**Done.** The whole non-negativity guarantee is one application of the triangle inequality. (And note  $h(v) = \delta(s, v) \leq 0$  always, since the 0-weight edge  $s \rightarrow v$  is one candidate path.)

**Why the new source  $s$  is necessary** (Exercise 23.3-2 / 23.3-6): to guarantee **every** vertex is reachable, so  $h$  is finite everywhere and Bellman-Ford sees every negative cycle. Adding  $s$  cannot change any  $\delta(u, v)$  for  $u, v \in V$ , because  $s$  has **no incoming edges** and therefore lies on no  $u \rightsquigarrow v$  path. If  $G$  happens to be strongly connected, any vertex works as  $s$ .

```

JOHNSON( $G, w$ )
1  build  $G'$ :  $V' = V \cup \{s\}$ ,  $E' = E \cup \{(s, v) : v \in V\}$ ,  $w(s, v) = 0$ 
2  if BELLMAN-FORD( $G', w, s$ ) == FALSE
3      report "negative-weight cycle"; return
4  for each  $v \in V'$ :  $h(v) = \delta(s, v)$ 
5  for each  $(u, v) \in E'$ :  $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
6  for each  $u \in V$ 
7      run DIJKSTRA( $G, \hat{w}, u$ ) to get  $\hat{\delta}(u, v)$  for all  $v$ 
8      for each  $v \in V$ 
9           $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$ 
    (23.11)
10 return  $D$ 

```

→ C++ implementation: [A6 JOHNSON](#)

**Line 9 undoes the reweighting.** By Lemma 23.1,  $\hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$ , so  $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$ . **Note the sign flip —  $+h(v) - h(u)$ , the reverse of the edge formula.** Getting this backwards is the classic Johnson bug.

## Complexity

| STEP                                                | COST                                         |
|-----------------------------------------------------|----------------------------------------------|
| Build $G'$                                          | $\Theta(V + E)$                              |
| Bellman-Ford on $G'$ ( $V+1$ vertices, $E+V$ edges) | $O(VE)$                                      |
| Reweight all edges                                  | $\Theta(E)$                                  |
| $V \times$ Dijkstra (Fibonacci heap)                | $V \cdot O(V \lg V + E) = O(V^2 \lg V + VE)$ |
| $V \times$ Dijkstra (binary heap)                   | $V \cdot O(E \lg V) = O(VE \lg V)$           |
| Undo reweighting                                    | $\Theta(V^2)$                                |

**Total:**  $O(V^2 \lg V + VE)$  with Fibonacci heaps,  $O(VE \lg V)$  with binary heaps. The Dijkstra phase dominates; the one Bellman-Ford run is a  $O(VE)$  rounding error. **Space**  $O(V^2)$  for the output (unavoidable) plus  $O(V + E)$ .

**When to pick which:**  $E \approx V \Rightarrow$  Johnson is  $O(V^2 \lg V)$  vs Floyd–Warshall's  $\Theta(V^3)$  — a huge win.  $E \approx V^2 \Rightarrow$  Johnson is  $O(V^3 \lg V)$  vs  $\Theta(V^3)$  — Floyd–Warshall wins, and by more than the  $\lg$  suggests because of constants. **Crossover in practice is well below  $E = V^2$** ; the measurement above put it comfortably between  $E = 1596$  and  $E = 95,767$  at  $V = 400$ .

## C++ Implementation

```
bool johnson(const Graph& g, vector<vector<long long>>& D) {
    const int n = g.n;
    Graph gp(n + 1); // G' =
    G + super-source s
    for (const Edge& e : g.edges) gp.addEdge(e.u, e.v, e.w);
    for (int v = 0; v < n; ++v) gp.addEdge(n, v, 0);

    BellmanFordResult bf = bellmanFord(gp, n);
    if (!bf.ok) return false; //
    negative-weight cycle

    vector<long long> h(bf.r.d.begin(), bf.r.d.begin() + n); //
    h(v) = δ(s, v) ≤ 0

    Graph gh(n); //
    ŵ(u,v) = w(u,v) + h(u) - h(v) ≥ 0
    for (const Edge& e : g.edges) gh.addEdge(e.u, e.v, e.w + h[e.u]
    - h[e.v]);

    D.assign(n, vector<long long>(n, INF));
    for (int u = 0; u < n; ++u) {
        SSSP r = dijkstra(gh, u);
        for (int v = 0; v < n; ++v)
            if (r.d[v] != INF) D[u][v] = r.d[v] + h[v] - h[u]; //
    }
    undo the reweighting
    return true;
}
```



***Empirical check.** Johnson reproduces CLRS Figure 23.4's matrix exactly. Over 400 random DAGs with weights in  $[-20, 20]$  and 454 random cyclic graphs with negative edges (no negative cycles), `johnson` agreed with Floyd–Warshall on **every entry of every matrix**. And Lemma 23.1's two guarantees were verified directly on 108 graphs:  $\hat{w}(u, v) \geq 0$  for **every edge**, and  $h(v) \leq 0$  for **every vertex**.*

**Johnson is worth studying even if you never implement it**, because it is the cleanest example in either book of a **three-technique composition**: difference constraints (§22.4) supply the reweighting scheme, Bellman-Ford (§22.1) solves for  $h$ , and Dijkstra (§22.3) exploits the result. The same "reweight so a faster algorithm becomes legal" pattern reappears in **min-cost max-flow** (M16), where Johnson's potentials are exactly what makes successive-shortest-paths run Dijkstra instead of Bellman-Ford.

## Outside / Engineering Context

*Not from Skiena or CLRS ch. 22–23; included because these come up constantly in interviews and in production routing code.*

| TECHNIQUE                     | IDEA                                                                                                                                                                                                                                                                                 | COST                             | WHEN                                                                  |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------|
| <b>A*</b>                     | Dijkstra with priority $d[v] + h(v)$ for an <b>admissible, consistent</b> heuristic $h$ ( $h(v) \leq \delta(v, t)$ , $h(u) \leq w(u, v) + h(v)$ ). With a consistent $h$ , A* is exactly Dijkstra on reweighted edges $w(u, v) + h(v) - h(u)$ — <b>literally Johnson's formula</b> . | same bound, far fewer expansions | single-pair with geometry (maps, games); $h$ = straight-line distance |
| <b>Bidirectional Dijkstra</b> | search forward from $s$ and backward from $t$ ; stop on a correct meeting condition (not merely "they touched")                                                                                                                                                                      | $\sim\sqrt{}$ the explored area  | single-pair, no heuristic available                                   |
| <b>0-1 BFS</b>                | weights only 0 or 1: replace the heap with a <b>deque</b> , push-front for 0, push-back for 1                                                                                                                                                                                        | $O(V + E)$                       | grids with free/costly moves — extremely common in CP                 |
| <b>Dial's algorithm</b>       | integer weights bounded by $c$ : bucket queue with $c \cdot V + 1$ buckets                                                                                                                                                                                                           | $O(E + VC)$                      | small integer weights                                                 |
| <b>k-ary heap</b>             | tune EXTRACT-MIN $O(k \log_k V)$ vs                                                                                                                                                                                                                                                  | $O(E)$                           | CLRS                                                                  |

|                                             |                                                                                                                                    |                                                         |                                                                                                |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------|
|                                             | DECREASE-KEY $O(\log_k V)$ ; $k = E/V$<br>balances them                                                                            | $\log_{\{E/V\}} V$                                      | Problem 23-2: matches Fibonacci-heap bounds on $\epsilon$ -dense graphs without the complexity |
| <b>Contraction hierarchies / hub labels</b> | heavy preprocessing, then constant-scale queries in microseconds                                                                   | preprocessing $\sim O(E \lg V)$ , query $\sim O(\lg V)$ | this is what real map services actually run                                                    |
| <b>Yen's algorithm</b>                      | $k$ shortest <b>loopless</b> paths, by repeatedly forbidding spur edges                                                            | $O(k V (E + V \lg V))$                                  | routing alternatives                                                                           |
| <b>Eppstein's algorithm</b>                 | $k$ shortest paths allowing loops                                                                                                  | $O(E + V \lg V + k)$                                    | theory + some CP                                                                               |
| <b>Viterbi</b>                              | max-probability state sequence in an HMM = <b>shortest path in a DAG</b> with $-\log$ weights                                      | $\Theta(T \cdot S^2)$                                   | speech, handwriting, decoding — see the war story                                              |
| <b>Currency arbitrage</b>                   | maximize $\prod \text{rates} \Rightarrow$ minimize $\sum (-\log \text{rate}) \Rightarrow$ <b>profitable cycle = negative cycle</b> | $O(VE)$<br>Bellman-Ford                                 | the canonical negative-cycle interview question                                                |
| <b>Minimum mean cycle (Karp)</b>            | the cycle minimizing $w(c)/ c $ ; a DP over $d_k(v) = \min \text{weight of a } k\text{-edge walk}$                                 | $O(VE)$                                                 | min-cost-to-time-ratio problems                                                                |

**The log trick is worth its own line.**  $\prod x_i$  is monotone in  $\sum \log x_i$ , so **any multiplicative path objective becomes additive under logs**. Maximize a product of probabilities  $\Rightarrow$  minimize a sum of  $-\log p$  (all non-negative since  $p \leq 1$ , so **Dijkstra applies**). Maximize a product of exchange rates  $\Rightarrow$  minimize  $\sum -\log(\text{rate})$ , where rates  $> 1$  give negative weights, so **Bellman-Ford applies and a negative cycle is an arbitrage**. Same trick as M14's minimum-product spanning tree.

## War Story: Dialing for Documents (Skiena 8.4)

The point of this story is not telephones. It is: **a problem with no distances, no map, and no graph in the statement was solved by building a graph and running a shortest-path algorithm.**

**The setup.** On an old telephone keypad each digit carries three letters. Typing one keystroke per letter is ambiguous: **2** could be A, B or C. Skiena's claim, made to an unimpressed tour guide at Periphonics: *"I'll bet that we could reconstruct English text correctly if it was typed in a telephone at one keystroke per letter."*

### Three failures before the graph.

1. **Trigram frequencies** in a sliding window. *"The trigram statistics did a decent job of translating it into gibberish, but a terrible job of transcribing English."* No knowledge of words.
2. **Dictionary + trigrams for the rest.** But collisions are brutal — *"the code string '22737' collides with eleven distinct English words, including cases, cares, cards, capes, caper, and bases."*
3. **Dictionary + word frequency.** *"This still made too many mistakes."* No grammar.

**The graph formulation.** Working with Harald Rau and grammatical statistics from the Brown Corpus:

*"Think of a sentence as a series of tokens, each representing a word in the sentence. Each token has a list of words from the dictionary that match it. How can we choose which one is right? Each possible sentence interpretation can be thought of as a path in a graph. Each vertex of this graph is one word from the complete set of possible word choices. There will be an edge from each possible choice for the **i**th word to each possible choice for the **(i+1)**st word. The cheapest path across this graph defines the best interpretation of the sentence."*

And then Harald's objection, which is the actual lesson:

*"But all the paths look the same. They have the same number of edges." — "Exactly! The cost of an edge will reflect how likely it is that we will travel through the given pair of words. Perhaps we can count how often that pair of words occurred together in previous texts. Or we can weigh them by the part of speech of each word. Maybe nouns don't like to be next to nouns as much as they like being next to verbs."*  
*"How can we factor [word frequency] into things?" — "We can pay a cost for walking through a particular vertex that depends upon the frequency of the word."*

**Notice what just happened.** The graph is a **layered DAG** (layer  $i$  = candidate words for token  $i$ ), so the shortest path is  $\Theta(V+E)$  by **DAG-SHORTEST-PATHS** — no Dijkstra needed, and negative weights would be fine. Edge costs encode **transition** likelihood (bigrams, part-of-speech compatibility); vertex costs encode **emission**/prior likelihood, folded into edges by exactly the transformation of Skiena's earlier "Stop and Think." **This is a hidden Markov model, and the shortest path is the Viterbi decode.**

**The result:** *"we typically guessed about 99% of all characters correctly,"* reconstructing the Gettysburg Address with a handful of errors (BATTLEFIELD OF THAT **WAS**; FINAL **SERVING** PLACE; WHO HERE **HAVE** THEIR LIVES). Periphonics licensed it. *"The reconstruction time was faster than anyone can type text in on a phone keypad."*

**The takeaway, in Skiena's words:** *"The constraints for many pattern recognition problems can be naturally formulated as shortest-path problems in graphs... Despite the fancy name, the Viterbi algorithm is basically solving a shortest-path problem on a DAG. **Hunting for a graph formulation to solve your problem is often the right idea.**"\**

---

## Recognition Patterns

**Reach for shortest paths when you see:**

| SIGNAL                                                                    | ALGORITHM                                                                               |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| "minimum cost / time / distance to get from ... to ..."                   | single-source                                                                           |
| <b>unweighted</b> graph, or all weights equal                             | <b>BFS</b> (M13) — don't reach for Dijkstra                                             |
| weights only 0 and 1                                                      | <b>0-1 BFS</b> with a deque                                                             |
| non-negative weights, one source                                          | <b>Dijkstra</b>                                                                         |
| any negative weight present                                               | <b>Bellman-Ford</b>                                                                     |
| "is there an arbitrage / a profitable loop / an inconsistency"            | Bellman-Ford <b>negative-cycle detection</b>                                            |
| the graph is a <b>DAG</b> (dependencies, stages, time steps, layers)      | <b>DAG-shortest-paths</b> — linear, and negatives are fine                              |
| "longest path" / "critical path" / "maximum score" — <b>on a DAG</b>      | negate and run DAG-shortest-paths                                                       |
| "longest path" on a general graph                                         | <b>NP-hard</b> (M19) — say so                                                           |
| all pairs, dense, $v \leq 500$                                            | <b>Floyd-Warshall</b>                                                                   |
| all pairs, sparse, negative weights                                       | <b>Johnson</b>                                                                          |
| all pairs, sparse, non-negative                                           | just run $v \times$ Dijkstra                                                            |
| "can vertex $i$ reach vertex $j$ " for all pairs                          | <b>transitive closure</b> (bitset Floyd-Warshall)                                       |
| "diameter" / "center" / "eccentricity"                                    | all-pairs, then reduce over the matrix                                                  |
| constraints of the form $x_j - x_i \leq b$                                | <b>difference constraints</b> $\rightarrow$ constraint graph $\rightarrow$ Bellman-Ford |
| maximize a <b>product</b> along a path                                    | take $-\log$ , minimize the sum                                                         |
| costs on <b>vertices</b>                                                  | move the cost onto in-edges                                                             |
| bounded number of special moves ( $\leq k$ refuels, teleports, discounts) | <b>layered graph</b> , $k+1$ copies                                                     |
| state beyond position matters (fuel, parity, keys held, last move)        | vertices = (position, state)                                                            |
| "shortest path using exactly $k$ edges"                                   | $(\min, +)$ matrix power $w^k$ by repeated squaring, $\Theta(v^3 \lg k)$                |
| maximize the <b>minimum</b> edge on a path (widest path)                  | $(\max, \min)$ semiring, or MST (M14)                                                   |
| best interpretation of a sequence under local costs                       | <b>layered DAG / Viterbi</b>                                                            |

## Common Mistakes

1. **Using Dijkstra with negative edges.** With a `visited` set it is *wrong*; without one it is *slow* (measured  $\Theta(V^2)$  pops on  $\Theta(V)$  edges). Neither is acceptable. Use Bellman-Ford.
2. **Confusing "negative edge" with "negative cycle."** Negative edges are a solved problem. Negative cycles mean *no answer exists*.
3. **Relaxing out of an unreachable vertex.** `INF + (-5) < INF` is true. Always guard `if (d[u] == INF) continue;`.
4. **Overflow.** `INT_MAX + w` wraps around. Use `long long` and `INF = LLONG_MAX / 4`.
5. **Floyd–Warshall with `k` not outermost.** Silent wrong answers. `k` is the DP stage; `i, j` are the table.
6. **0 for "no edge" in a weight matrix.** Skiena's warning: a 0 is a **free edge**, not a missing one. Use `INF`.
7. **Forgetting `d[i][i] = 0`** in the Floyd–Warshall input matrix.
8. **Getting Johnson's un-reweighting backwards.** Edges use `+h(u) - h(v)`; distances use `+h(v) - h(u)`.
9. **Reweight by subtracting `min w`.** Ex 23.3-4 — it penalizes long paths and changes the answer.
10. **Forgetting the stale-entry check** in the lazy-heap Dijkstra, or writing `less<>` instead of `greater<>`.
11. **Assuming the SPT is the MST.** Different objectives, usually different trees.
12. **Using Dijkstra where BFS suffices.** On an unweighted graph BFS is  $\Theta(V+E)$ ; Dijkstra adds a `lg V` for nothing.
13. **Reconstructing paths without checking reachability.** Walking  `$\pi$`  from an unreachable `v` gives an empty or garbage path.
14. **Only detecting a negative cycle, when the problem asked you to print one.** Learn the `|V|`-steps-back trick.
15. **Trying to compute a single-pair distance "faster."** There is no known asymptotic improvement; only heuristics ( $A^*$ , bidirectional).

---

## Complexity Summary

Single-source:

| ALGORITHM                        | TIME                                         | SPACE                   | NEGATIVE EDGES | NEGATIVE CYCLES      |
|----------------------------------|----------------------------------------------|-------------------------|----------------|----------------------|
| <b>BFS</b> (unweighted)          | $\Theta(V + E)$                              | $\Theta(V)$             | n/a            | n/a                  |
| <b>0-1 BFS</b> (deque)           | $\Theta(V + E)$                              | $\Theta(V)$             | no             | n/a                  |
| <b>DAG-shortest-paths</b>        | $\Theta(V + E)$                              | $\Theta(V)$             | <b>yes</b>     | impossible (acyclic) |
| <b>Dijkstra</b> , array          | $\Theta(V^2 + E)$                            | $\Theta(V)$             | <b>no</b>      | no                   |
| <b>Dijkstra</b> , binary heap    | $O((V+E) \lg V)$                             | $\Theta(V + E)$<br>lazy | <b>no</b>      | no                   |
| <b>Dijkstra</b> , Fibonacci heap | $O(V \lg V + E)$                             | $\Theta(V)$             | <b>no</b>      | no                   |
| <b>Dial</b> (weights $\leq C$ )  | $O(E + VC)$                                  | $\Theta(V + C)$         | no             | no                   |
| <b>Bellman-Ford</b>              | $\Theta(VE)$ , $\Theta(hE)$ w/<br>early exit | $\Theta(V)$             | <b>yes</b>     | <b>detects</b>       |
| <b>SPFA</b> (queued BF)          | $\Theta(VE)$ worst, fast<br>typical          | $\Theta(V)$             | <b>yes</b>     | detects              |

**All-pairs:**

| ALGORITHM                               | TIME                                              | SPACE                 | NEGATIVE EDGES | NOTES                                                                 |
|-----------------------------------------|---------------------------------------------------|-----------------------|----------------|-----------------------------------------------------------------------|
| $V \times$ Dijkstra (heap)              | $O(VE \lg V)$                                     | $\Theta(V^2)$         | no             | best for sparse non-negative                                          |
| $V \times$ Bellman-Ford                 | $O(V^2E)$                                         | $\Theta(V^2)$         | yes            | the baseline to beat                                                  |
| SLOW-APSP<br>( $(\min, +)$ powers)      | $\Theta(V^4)$                                     | $\Theta(V^2)$         | yes            | pedagogical                                                           |
| FASTER-APSP<br>(repeated squaring)      | $\Theta(V^3 \lg V)$                               | $\Theta(V^2)$         | yes            | but gives "exactly $k$ edges" for free                                |
| Floyd-Warshall                          | $\Theta(V^3)$                                     | $\Theta(V^2)$         | yes            | tightest constants; <b>detects negative cycles via</b> $d[i][i] < 0$  |
| Transitive closure<br>( $(V, \wedge)$ ) | $\Theta(V^3), \Theta(V^3/64)$<br>bitset           | $\Theta(V^2)$<br>bits | n/a            | Warshall                                                              |
| Johnson                                 | $O(V^2 \lg V + VE)$<br>Fib, $O(VE \lg V)$<br>heap | $\Theta(V^2)$         | yes            | best for <b>sparse</b> with negatives; <b>detects negative cycles</b> |

Where the numbers come from (say these, don't memorize them):

- Bellman-Ford  $\Theta(VE)$ :  $|V| - 1$  passes because a simple path has at most  $|V| - 1$  edges;  $|E|$  relaxations per pass.
- Dijkstra:  $V$  extract-mins (each vertex is finalized once) +  $\leq E$  decrease-keys (each edge is relaxed once). Plug the queue's costs into  $V \cdot T_{\text{extract}} + E \cdot T_{\text{decrease}}$ .
- Floyd-Warshall  $\Theta(V^3)$ :  $V$  stages  $\times V^2$  table entries  $\times O(1)$ .
- Johnson: one  $O(VE)$  Bellman-Ford +  $V$  Dijkstras. The Dijkstras dominate.

## One-Page Recall

- $\delta(u, v)$  = min weight over all  $u \rightsquigarrow v$  paths;  $\infty$  if unreachable,  $-\infty$  if a negative-weight cycle sits on some  $u \rightsquigarrow v$  walk.
- **Lemma 22.1**: subpaths of shortest paths are shortest paths. This is the optimal substructure everything rests on.



- Shortest paths may be assumed **simple**, hence  $\leq |V| - 1$  edges. Positive cycles never help, zero cycles never need to be used, negative cycles destroy the problem.
- **RELAX**( $u, v, w$ ): **if**  $v.d > u.d + w$  **then**  $v.d = u.d + w$ ;  $v.\pi = u$ . Every algorithm here is "relax edges in a good order." Always guard against relaxing out of an  $\infty$  vertex.
- **Six properties**: triangle inequality  $\delta(s, v) \leq \delta(s, u) + w(u, v)$ ; upper-bound  $v.d \geq \delta$ ; no-path; convergence; **path-relaxation** (relax a shortest path's edges in order  $\Rightarrow$  converged); predecessor-subgraph ( $G_\pi$  is an SPT).
- **Bellman-Ford**:  $|V| - 1$  passes over all edges, then one more pass as the negative-cycle test.  $\Theta(V E)$ . Correct because  $|V| - 1$  passes contain every simple shortest path as an in-order subsequence (Lemma 22.15). Add the **early exit**. Negative-cycle proof = *sum the inequalities around the cycle, the  $d$ 's cancel, get  $0 \leq w(c) < 0$* .
- To **print** a negative cycle: after  $|V|$  passes take a vertex that still relaxed, walk  $\pi$  back  $|V|$  times (guaranteed inside the cycle), then walk until repeat. To mark  $-\infty$ : flood-fill forward from every still-relaxable endpoint.
- **DAG-shortest-paths**: topological order, one pass.  $\Theta(V + E)$ , **negative weights fine**. Negate for **longest path** / **critical path** / **PERT** — the only setting where longest path is easy.
- **Dijkstra**: greedily finalize the closest unfinalized vertex. **Needs**  $w \geq 0$ .  $\Theta(V^2)$  array /  $O(E \lg V)$  binary heap /  $O(V \lg V + E)$  Fibonacci heap — from  $V$  extract-mins +  $E$  decrease-keys.
- **Theorem 22.6's squeeze**:  $\delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d = \delta(s, y)$ . **Non-negativity is used in exactly one step** —  $\delta(s, y) \leq \delta(s, u)$  for  $y$  earlier on the path.
- **Dijkstra vs Prim: the only difference is the key**. Prim ranks by  $w(u, v)$ , Dijkstra by  $d[u] + w(u, v)$ . Skiena: *four changed lines*.
- **Lazy decrease-key**: push on every improvement, **if** ( $du > d[u]$ ) **continue**; on pop. Same bound, no custom heap,  $\Theta(E)$  memory.
- **Negative edges + Dijkstra**: with a **visited** set  $\rightarrow$  *wrong*; without  $\rightarrow$  *correct but Bellman-Ford-slow*.
- **Design graphs, not algorithms** (Skiena). Vertex costs  $\rightarrow$  in-edge weights.  $k$  special moves  $\rightarrow k+1$  layers. Extra state  $\rightarrow$  product graph. Products  $\rightarrow$  logs.
- **Difference constraints**  $x_j - x_i \leq b$ : edge  $(v_i, v_j)$  of weight  $b$ , plus super-source  $v_0$  with  $0$ -edges. Feasible  $\iff$  no negative cycle, and  $x_i = \delta(v_0, v_i)$  is a solution — because **the triangle inequality is the constraint**.  $O(nm)$ .

- **(min, +) = tropical semiring.** `EXTEND-SHORTEST-PATHS` is matrix multiplication in it;  $L^{(n-1)} = W^{(n-1)}$  is the answer; repeated squaring gives  $\Theta(V^3 \lg V)$  and, more usefully, "shortest walk of exactly  $k$  edges" in  $\Theta(V^3 \lg k)$ . Swap the semiring for closure  $(\vee, \wedge)$ , path counts  $(+, \times)$ , bottlenecks  $(\max, \min)$ .
  - **Floyd–Warshall:**  $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ ,  $k$  = "intermediates allowed from  $\{1..k\}$ ".  **$k$  OUTERMOST.**  $\Theta(V^3)$ ,  $\Theta(V^2)$  space in place, tiny constants, best on dense graphs. Correct because when  $k$  is on the path it is an **endpoint**, not an intermediate, of each half.
  - **$a[i][i] < 0$  after Floyd–Warshall  $\iff i$  is on a negative cycle.** Also gives diameter, center, transitive closure (bitset it:  $64\times$ ).
  - **Johnson:**  $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$  **telescopes**, so path weights shift by  $h(u) - h(v)$  regardless of length, and cycle weights don't change at all. Take  $h(v) = \delta(s, v)$  from a super-source via Bellman-Ford; the **triangle inequality** then gives  $\hat{w} \geq 0$ ; run  $V$  Dijkstras; undo with  $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$ .  $\Theta(V^2 \lg V + VE)$ . **Do not** reweight by subtracting  $\min w$ .
  - **Choosing, in five seconds:** unweighted  $\rightarrow$  BFS. DAG  $\rightarrow$  topological. One source,  $w \geq 0 \rightarrow$  Dijkstra. Any negative weight  $\rightarrow$  Bellman-Ford. All pairs, dense  $\rightarrow$  Floyd–Warshall. All pairs, sparse  $\rightarrow$  Johnson (or  $V \times$  Dijkstra if  $w \geq 0$ ).
- 

## Practice — where to drill this module

| IDEA IN THIS MODULE                   | PROBLEM                                                                                            | WHY IT'S THE RIGHT DRILL                                                                                         |
|---------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Dijkstra, plain                       | <a href="#">743 · Network Delay Time</a>                                                           | the minimal complete Dijkstra: build the graph, relax, take the max                                              |
| Dijkstra with extra state             | <a href="#">787 · Cheapest Flights Within K Stops</a>                                              | the state is <code>(city, stopsUsed)</code> ; also solvable by <code>k+1</code> rounds of Bellman-Ford — do both |
| Dijkstra with a different semiring    | <a href="#">1514 · Path with Maximum Probability</a>                                               | maximise a <b>product</b> ; take <code>-log</code> and it is ordinary Dijkstra ( <a href="#">M02</a> )           |
| 0-1 BFS                               | <a href="#">1368 · Minimum Cost to Make at Least One Valid Path in a Grid</a>                      | weights in <code>{0,1}</code> $\Rightarrow$ a <b>deque</b> replaces the heap and it is $\Theta(V+E)$             |
| Floyd–Warshall, whole                 | <a href="#">1334 · Find the City With the Smallest Number of Neighbors at a Threshold Distance</a> | <code>n ≤ 100</code> , so the $\Theta(V^3)$ triple loop is <i>intended</i> — <a href="#">A5</a> as a submission  |
| DAG shortest/longest paths            | <a href="#">329 · Longest Increasing Path in a Matrix</a>                                          | the graph is acyclic by construction, so topological order works and longest path is easy                        |
| Bitmask state + BFS                   | <a href="#">847 · Shortest Path Visiting All Nodes</a>                                             | <code>(node, visitedMask)</code> ; the Held–Karp shape                                                           |
| "Design the graph, not the algorithm" | <a href="#">1584 · Min Cost to Connect All Points</a>                                              | contrast with <a href="#">M14</a> : MST minimises the <b>total</b> , shortest paths minimise <b>each path</b>    |

**Beyond LeetCode.** [CSES Problem Set — Graph Algorithms](#) has a dedicated shortest-path run (Dijkstra, Bellman-Ford with negative cycles, Floyd–Warshall) and is the best single ladder for this module. [Codeforces shortest paths tag](#) · [graphs tag](#).

**The drill that matters here** is the five-second decision: *unweighted*  $\rightarrow$  *BFS*; *DAG*  $\rightarrow$  *topological order*; `w ≥ 0`  $\rightarrow$  *Dijkstra*; *any negative weight*  $\rightarrow$  *Bellman-Ford*; *all pairs and dense*  $\rightarrow$  *Floyd–Warshall*; *all pairs and sparse*  $\rightarrow$  *Johnson*. Say it out loud before writing anything.

## C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., ch. 9 and §6.9.

## 1. `INF` as a finite sentinel, and the two ways it bites

```
const long long PATH_INF = LLONG_MAX / 4;    // NOT LLONG_MAX
```

Two separate hazards, and they need two separate defences:

- **Overflow.** `INF + w` with `INF == LLONG_MAX` is signed overflow — **undefined behaviour**, not saturation. Dividing by 4 leaves headroom for an addition and a comparison.
- **Relaxing out of unreachable.** With a negative `w`, `INF + w < INF` is *true*, so an unreachable vertex would "improve" a neighbour. **The guard `if (d[u] == INF)` `return false;` is not optional** — it is the single most common source of wrong answers in negative-weight code, and it is silent.

## 2. `priority_queue` for Dijkstra: min-heap, and `pair` order matters

```
void dijkstraQueueShape() {  
    using Item = pair<long long,int>;           //  
    (distance, vertex)  
    priority_queue<Item, vector<Item>, greater<Item>> pq;  //  
    greater<> => MIN-heap  
    (void)pq;  
}
```

`priority_queue` is a **max**-heap by default; `greater<>` inverts it, and the comparator is the **third** template argument so the container must be spelled out. `pair` compares lexicographically, so **distance must be `.first`** — reversing to `(vertex, distance)` orders by vertex id and produces a wrong answer that runs to completion.

## 3. The lazy decrease-key idiom — the same one as [M14](#)

`std::priority_queue` has no `decrease-key`, which `DIJKSTRA` line 11 wants. Push a duplicate on every improvement and skip stale entries on pop:

```
// if (du > d[u]) continue;    // this entry is out of date; a  
better one exists
```

$O(E \lg E) = O(E \lg V)$  — the same bound,  $\Theta(E)$  memory instead of  $\Theta(V)$ , and no custom heap. **This is the default.** Note that it also means there is **no visited set**, which has a consequence for negative edges — see A4.

#### 4. `vector<vector<long long>>` for the all-pairs matrix

Floyd–Warshall touches  $V^3$  cells with a fixed scan pattern, so locality dominates. `vector<vector<long long>>` matches the recurrence's `d[i][j]` and is what the appendix uses; for large  $V$ , one flat `vector<long long>` of size  $n*n$  indexed `d[i*n + j]` is measurably faster (M03 toolkit §7). The  $\Theta(V^2)$  output is unavoidable either way.

#### 5. `unsigned __int128` is not needed here, but `long long` everywhere is

Path weights accumulate over up to  $V-1$  edges.  $V = 10^5$  with weights to  $10^9$  gives  $10^{14}$  — well past `int`. Every distance, every accumulator, every matrix cell in this module is `long long`.

#### 6. Returning a struct, not a pair

```
struct PathResult { vector<long long> d; vector<int> pi; };
```

Every single-source algorithm produces **two** arrays — distances and predecessors — and they are meaningless apart. `.d` and `.pi` beat `.first` and `.second`, and adding a `bool ok` for the negative-cycle case later does not break existing call sites.

#### 7. Reverse iterators and `reverse` for path reconstruction

Predecessor chains are walked backwards and must be flipped:

```
void reconstructShape(vector<int>& path) {  
    reverse(path.begin(), path.end());    // <algorithm>, in place,  
    Theta(n)  
}
```

Always check reachability **before** walking `pi`: from an unreachable vertex the chain either terminates at `-1` without reaching the source, or (with a bug) loops forever.

---

## Appendix — C++ for Every Pseudocode Block

The body of this module already contains the *practical* implementations. These are the **literal** ones — line for line against the pseudocode, with the C++ decisions spelled out.

```
// Shared representation for the appendix.
const long long PATH_INF = LLONG_MAX / 4;      // toolkit 1

struct WEdge { int u, v; long long w; };

struct WGraph {
    int n = 0;
    vector<vector<pair<int, long long>>> adj;    // (neighbour,
weight)
    vector<WEdge> edges;                      // flat list, for
Bellman-Ford
    explicit WGraph(int n_) : n(n_), adj(n_) {}
    void addEdge(int u, int v, long long w) {
        adj[u].push_back({v, w});
        edges.push_back({u, v, w});
    }
};

// Every single-source algorithm returns these two arrays together
(toolkit 6).
struct PathResult {
    vector<long long> d;    // d[v] = shortest-path estimate
    vector<int> pi;        // pi[v] = predecessor, -1 = NIL
};
```

### A1 RELAX (and INITIALIZE-SINGLE-SOURCE)

*Pseudocode: Big Idea.*

```
// INITIALIZE-SINGLE-SOURCE(G, s)
void initSingleSource(const WGraph& g, int s, PathResult& r) {
    r.d.assign(g.n, PATH_INF);    // v.d = infinity
    r.pi.assign(g.n, -1);        // v.pi = NIL
    r.d[s] = 0;                  // s.d = 0
}
```

```

// RELAX(u, v, w) -- the ONE operation every algorithm in this
module is built
// from. "Is going through u a better way to reach v than what I
already have?"
//
// Returns whether the estimate actually improved. That single bool
is what
// makes Bellman-Ford's early exit, Dijkstra's push, and the
negative-cycle
// test each a one-liner.
bool relax(int u, int v, long long w, PathResult& r) {
    // THE GUARD (toolkit 1). Without it, an unreachable u with a
negative w
    // gives PATH_INF + w < PATH_INF, and the algorithm confidently
computes
    // distances to vertices it cannot reach. Silent, and always
wrong.
    if (r.d[u] == PATH_INF) return false;
    if (r.d[v] > r.d[u] + w) {           // 1  if v.d > u.d + w(u,v)
        r.d[v] = r.d[u] + w;           // 2      v.d = u.d +
w(u,v)
        r.pi[v] = u;                   // 3      v.pi = u
        return true;
    }
    return false;
}

// Walk pi backwards from v to s, then reverse (toolkit 7).
vector<int> extractPath(const PathResult& r, int s, int v) {
    if (r.d[v] == PATH_INF) return {}; // unreachable:
check FIRST
    vector<int> path;
    for (int x = v; x != -1; x = r.pi[x]) {
        path.push_back(x);
        if (x == s) break;
    }
    if (path.back() != s) return {}; // pi chain did
not reach s
    reverse(path.begin(), path.end());
    return path;
}

```

**Complexity.**  $\Theta(1)$  per relaxation;  $\Theta(V)$  to initialise.

**The name is backwards and CLRS says so:** you are *tightening* an estimate by *relaxing* the violated constraint  $v.d \leq u.d + w(u,v)$ . Do not fight it.

**The six properties that every correctness proof in this module is assembled from:** triangle inequality ( $\delta(s,v) \leq \delta(s,u) + w(u,v)$ ), upper-bound ( $v.d \geq \delta(s,v)$ , and once equal it never changes), no-path, convergence, **path-relaxation** (relax a shortest path's edges *in order* and its endpoint converges, whatever else happened in between), and predecessor-subgraph. **Path-relaxation is the one that does the work** — every algorithm below is just a different way of guaranteeing that ordering.

## A2 BELLMAN-FORD

*Pseudocode: Part 2.*

```
struct BellmanFordOutput {
    bool ok = true;           // false <=> a negative-weight cycle is
    REACHABLE from s
    PathResult r;
};

BellmanFordOutput bellmanFord(const WGraph& g, int s) {
    BellmanFordOutput out;
    initSingleSource(g, s, out.r);           // 1

    for (int i = 1; i < g.n; ++i) {         // 2 |V|
- 1 passes
        bool changed = false;
        for (const WEdge& e : g.edges)     // 3 for
each edge
            if (relax(e.u, e.v, e.w, out.r)) changed = true; // 4
RELAX

        // EARLY EXIT -- not in the pseudocode, always worth
writing. If a whole
        // pass changes nothing, no later pass will either. This
turns Theta(VE)
        // into Theta(hE) where h is the largest hop-count of any
shortest path,
        // which on real graphs is far below V.
        if (!changed) break;
    }
}
```



```

    // 5-7 The  $|V|$ -th pass IS the test: if anything can still be
    relaxed after
    //  $|V|-1$  passes, some shortest "path" uses  $\geq |V|$  edges, which
    means it
    // repeats a vertex, which means there is a negative-weight
    cycle.
    for (const WEdge& e : g.edges)
        if (out.r.d[e.u] != PATH_INF && out.r.d[e.v] > out.r.d[e.u]
+ e.w) {
            out.ok = false; // 7
    return FALSE
        break;
    }
    return out; // 8
return TRUE
}

const long long PATH_NEG_INF = LLONG_MIN / 4;

// Exercise 22.1-4: mark the vertices whose true shortest-path
weight is -infinity.
// Run AFTER bellmanFord. A vertex is -infinity iff it is reachable
from a
// negative cycle that is itself reachable from s -- so: find the
still-relaxable
// endpoints, then flood forward from them.
void markNegativeInfinity(const WGraph& g, PathResult& r) {
    vector<char> bad(g.n, 0);
    vector<int> stack;
    for (const WEdge& e : g.edges)
        if (r.d[e.u] != PATH_INF && r.d[e.v] > r.d[e.u] + e.w &&
!bad[e.v]) {
            bad[e.v] = 1; stack.push_back(e.v);
        }
    while (!stack.empty()) { //
everything downstream
        int u = stack.back(); stack.pop_back();
        for (const auto& [v, w] : g.adj[u]) {
            (void)w;
            if (!bad[v]) { bad[v] = 1; stack.push_back(v); }
        }
    }
}

```

```

    }
    for (int v = 0; v < g.n; ++v) if (bad[v]) r.d[v] =
PATH_NEG_INF;
}

// Exercise 22.1-7: PRINT an actual negative cycle, not merely
detect one.
vector<int> findNegativeCycle(const WGraph& g, int s) {
    PathResult r;
    initSingleSource(g, s, r);
    int x = -1;
    for (int i = 0; i < g.n; ++i) { // |V|
passes, not |V|-1
        x = -1;
        for (const WEdge& e : g.edges)
            if (relax(e.u, e.v, e.w, r)) x = e.v; //
remember the LAST change
        }
        if (x == -1) return {}; // nothing
relaxed: no cycle

        // x is reachable from a cycle via pi pointers, but may not be
ON it. The
        // pi-chain's prefix before the cycle has fewer than |V|
vertices, so
        // walking back |V| steps lands INSIDE the cycle for certain.
        for (int i = 0; i < g.n; ++i) x = r.pi[x];

        vector<int> cycle;
        for (int v = x;; v = r.pi[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end());
        return cycle; // v0 ==
vk
    }
}

```

**Complexity.**  $\Theta(V E)$  —  $|V|-1$  passes over  $|E|$  edges — or  $\Theta(h E)$  with the early exit.  
**Space**  $\Theta(V)$ , no recursion.

**Why  $|V| - 1$  and not some safety margin:** a shortest path may be assumed **simple**, so it has at most  $|V| - 1$  edges. Pass  $i$  relaxes the  $i$ -th edge of *every* shortest path, so after  $|V| - 1$  passes every shortest path has been relaxed in order — and the **path-relaxation property** then guarantees convergence. That is the entire correctness argument (Lemma 22.2  $\rightarrow$  Theorem 22.4).

**The negative-cycle proof is worth memorising verbatim**, because the same move appears three more times in this module. Suppose the check does *not* fire on a negative cycle  $c = \langle v_0, \dots, v_k \rangle$ ,  $v_0 = v_k$ . Then  $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$  for every  $i$ . Sum around the cycle: each  $d$  appears once on each side and they are **finite** (the cycle is reachable), so they cancel, leaving  $0 \leq w(c) < 0$ . Contradiction. ■ *(The same cancellation proves Theorem 22.9 for difference constraints and Lemma 23.1 for Johnson's reweighting.)*

**A negative cycle not reachable from  $s$  is harmless** and Bellman-Ford correctly returns `true`. Marking  $-\infty$  is a separate reachability computation, which is why it is a separate function.

## A3 DAG-SHORTEST-PATHS

*Pseudocode: Part 3.*

```
// Kahn topological order; empty result means the graph has a
// cycle.
vector<int> topoOrder(const WGraph& g) {
    vector<int> indeg(g.n, 0), order, q;
    for (const WEdge& e : g.edges) ++indeg[e.v];
    for (int v = 0; v < g.n; ++v) if (indeg[v] == 0)
        q.push_back(v);
    while (!q.empty()) {
        int u = q.back(); q.pop_back();
        order.push_back(u);
        for (const auto& [v, w] : g.adj[u]) { (void)w; if (--
            indeg[v] == 0) q.push_back(v); }
    }
    return (int)order.size() == g.n ? order : vector<int>{};
}

PathResult dagShortestPaths(const WGraph& g, int s) {
    PathResult r;
    initSingleSource(g, s, r);
}
```

// 2

```

    for (int u : topoOrder(g)) // 1,3 in
    TOPOLOGICAL order
        if (r.d[u] != PATH_INF) // skip
        unreachable vertices
            for (const auto& [v, w] : g.adj[u]) // 4
                relax(u, v, w, r); // 5
    // ONE pass. Every edge relaxed EXACTLY ONCE.
    return r;
}

// LONGEST path in a DAG -- the critical path / PERT computation.
// Negate every weight, run the same algorithm, negate the answers
back.
// This is legal ONLY because there are no cycles: negation in a
general graph
// manufactures negative cycles and destroys the problem.
PathResult dagLongestPaths(const WGraph& g, int s) {
    WGraph h(g.n);
    for (const WEdge& e : g.edges) h.addEdge(e.u, e.v, -e.w);
    PathResult r = dagShortestPaths(h, s);
    for (long long& x : r.d) if (x != PATH_INF) x = -x;
    return r;
}

```

**Complexity.**  $\Theta(V + E)$  — topological sort plus one pass. **Space**  $\Theta(V)$ . **This is the fastest shortest-path algorithm in the module, and it tolerates negative weights.**

**Why (Theorem 22.5):** for any shortest path  $\langle v_0, \dots, v_k \rangle$ , the vertices appear in that relative order in *any* topological order (Lemma 20.11 / M13). The algorithm relaxes all of  $u$ 's out-edges when it reaches  $u$ , so it relaxes the path's edges **in order** — and the path-relaxation property finishes the argument. The proof never used  $w \geq 0$ , which is why negative weights are fine.

**Longest path is NP-hard in a general graph and linear in a DAG.** Whenever a problem's dependency structure is acyclic — jobs with prerequisites, stages in a pipeline, a grid you only move right/down through, the subproblem graph of a DP — this is the tool.

## A4 DIJKSTRA

*Pseudocode: Part 4.*

```

// REQUIRES w(u,v) >= 0 for every edge.
PathResult dijkstra(const WGraph& g, int s) {
    PathResult r;
    initSingleSource(g, s, r); // 1

    using Item = pair<long long, int>; //
    (d[v], v) -- distance
    priority_queue<Item, vector<Item>, greater<Item>> Q; //
    FIRST (toolkit 2)
    Q.push({0, s}); // 4-5
    INSERT all vertices
    // (only s is pushed: a vertex enters the queue the first time
    its estimate
    // becomes finite, which is equivalent and avoids V pointless
    entries)

    while (!Q.empty()) { // 6
        auto [du, u] = Q.top(); Q.pop(); // 7 u
    = EXTRACT-MIN(Q)
        if (du > r.d[u]) continue; //
    STALE entry (toolkit 3)
        // 8 S = S union {u}. There is no explicit S: the stale
    check does its
        // job, because once d[u] is final no later entry can
    beat it.
        for (const auto& [v, w] : g.adj[u]) // 9
            if (relax(u, v, w, r)) // 10
                Q.push({r.d[v], v}); // 11
    DECREASE-KEY, lazily
    }
    return r;
}

// Theta(V^2) Dijkstra: no heap, linear scan for the minimum. The
RIGHT choice
// on a dense graph, and Skiena's implementation ([M14] toolkit 4
makes the same
// point for Prim).
PathResult dijkstraDense(const WGraph& g, int s) {
    PathResult r;
    initSingleSource(g, s, r);

```

```

    vector<char> done(g.n, 0); // the
explicit set S
    for (int iter = 0; iter < g.n; ++iter) {
        int u = -1;
        long long best = PATH_INF;
        for (int i = 0; i < g.n; ++i) //
EXTRACT-MIN by scan
            if (!done[i] && r.d[i] < best) { best = r.d[i]; u = i;
        }
        if (u == -1) break; // the
rest is unreachable
        done[u] = 1;
        for (const auto& [v, w] : g.adj[u]) relax(u, v, w, r);
    }
    return r;
}

```

**Complexity** — read the table as  $V \cdot T_{\text{extract}} + E \cdot T_{\text{decrease}}$ :

| PRIORITY QUEUE      | TOTAL                           | BEST WHEN               |
|---------------------|---------------------------------|-------------------------|
| array / linear scan | $\Theta(V^2 + E) = \Theta(V^2)$ | dense                   |
| binary heap (lazy)  | $O((V+E) \lg V) = O(E \lg V)$   | sparse — the default    |
| Fibonacci heap      | $O(V \lg V + E)$                | theory; large constants |

$V$  extract-mins because each vertex is finalised once;  $\leq E$  decrease-keys because each edge can improve a key once. **Fibonacci heaps were invented to make that second line  $O(1)$**  — Dijkstra is their design brief.

**Theorem 22.6, the squeeze.** Suppose  $u$  is the first vertex extracted with  $u.d \neq \delta(s, u)$ . Walk a shortest path  $s \rightsquigarrow u$  to the first vertex  $y$  not yet in  $S$ , with predecessor  $x \in S$ . Then:

$$\delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d = \delta(s, y)$$

—  $y$  precedes  $u$  on a shortest path; upper-bound property; EXTRACT-MIN chose  $u$  over  $y$ ; convergence property applied when  $(x, y)$  was relaxed. Everything collapses to equality, contradiction. ■

**Non-negativity is used in exactly one step:**  $\delta(s, y) \leq \delta(s, u)$ , which needs the segment  $y \rightsquigarrow u$  to have weight  $\geq 0$ . Knowing *which* line fails is what separates "Dijkstra doesn't work with negative edges" from an actual answer.

**And here is the subtlety worth carrying:** the lazy version above has **no visited set**, so if a negative edge later improves an already-popped vertex it is simply pushed again. That makes it *correct* on negative edges (absent negative cycles) and destroys the  $O(E \lg V)$  bound — it degenerates into a Bellman-Ford-flavoured algorithm. `dijkstraDense`, which *does* keep `done[]`, returns a **wrong answer** instead. **With a visited set: wrong. Without one: slow.** Use Bellman-Ford, or reweight with Johnson.

## A5 FLOYD-WARSHALL

*Pseudocode: Part 7.*

```
struct AllPairs {
    vector<vector<long long>> d;
    vector<vector<int>> pi;        // -1 = NIL
};

// W is the weight matrix: W[i][i] = 0, W[i][j] = w(i,j) or
// PATH_INF.
AllPairs floydWarshall(const vector<vector<long long>>& W) {
    const int n = (int)W.size();
    AllPairs a;
    a.d = W;
    a.pi.assign(n, vector<int>(n, -1));
    for (int i = 0; i < n; ++i)                // (23.7)
        predecessor init
            for (int j = 0; j < n; ++j)
                if (i != j && W[i][j] < PATH_INF) a.pi[i][j] = i;

    // k IS THE OUTERMOST LOOP. This is the one thing never to get
    // wrong.
    // d^(k)[i][j] depends on ALL of stage k-1, so only "for each
    // k, fill the
    // whole table" establishes the invariant "every path through
    // {0..k} found".
    // Writing for i / for j / for k compiles, runs in the same
    // time, and
    // silently returns wrong answers on some graphs.
    for (int k = 0; k < n; ++k)
```

```

        for (int i = 0; i < n; ++i) {
            if (a.d[i][k] == PATH_INF) continue;           // nothing
routes through k
            // Hoisting this test out of the inner loop is the
single most
            // valuable micro-optimisation here, and it removes the
            // PATH_INF + PATH_INF overflow risk as a bonus.
            for (int j = 0; j < n; ++j) {
                if (a.d[k][j] == PATH_INF) continue;
                long long through = a.d[i][k] + a.d[k][j];
                if (through < a.d[i][j]) {
                    a.d[i][j] = through;                  // (23.6)
                    a.pi[i][j] = a.pi[k][j];             // (23.8) --
pi[k][j], NOT k
                }                                           // and not
pi[i][k]
            }
        }
        return a;
    }
}

```

// Exercise 23.2-4: the superscripts are unnecessary -- update in place.

// During iteration k, d[i][k] and d[k][j] cannot change (d[k][k] == 0 given no

// negative cycles), so reading a partially-updated row is harmless.

// Theta( $n^3$ ) space becomes Theta( $n^2$ ).

```

vector<vector<long long>> floydWarshallInPlace(vector<vector<long
long>> d) {
    const int n = (int)d.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (d[i][k] != PATH_INF && d[k][j] != PATH_INF &&
                    d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
    return d;
}

```

// TRANSITIVE CLOSURE (23.9): the SAME recurrence with (min,+) replaced by



```

// (OR, AND). Booleans instead of longs, so it also bitset-
vectorises: pack row
// j into a bitset and the inner loop becomes `if (t[i][k]) t[i] |=
t[k];`,
// giving Theta(V^3 / 64).
vector<vector<char>> transitiveClosure(const vector<vector<char>>&
adj) {
    const int n = (int)adj.size();
    vector<vector<char>> t = adj;
    for (int i = 0; i < n; ++i) t[i][i] = 1;           // reflexive
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            if (t[i][k])
                for (int j = 0; j < n; ++j)
                    if (t[k][j]) t[i][j] = 1;
    return t;
}

void printAllPairsPath(const AllPairs& a, int i, int j,
vector<int>& out) {
    if (i == j) out.push_back(i);
    else if (a.pi[i][j] == -1) out.clear();             // no path
    else { printAllPairsPath(a, i, a.pi[i][j], out);
    out.push_back(j); }
}

```

**Complexity.**  $\Theta(V^3)$  time —  $\Theta$ , not  $o$ : no early exit, no data dependence, always exactly  $n^3$  iterations. Space  $\Theta(V^2)$  in place.

**The recurrence and the subtle step.**  $d^{(k)}[i][j]$  is the shortest  $i \rightarrow j$  path whose **intermediate** vertices all lie in  $\{0..k\}$ . Either  $k$  is not on the path — so  $d^{(k)} = d^{(k-1)}$  — or it is, and the path splits as  $i \rightsquigarrow k \rightsquigarrow j$ . **The crucial observation is that  $k$  is an endpoint of each half, not an intermediate**, so both halves have intermediates in  $\{0..k-1\}$  and are already computed. If you cannot say that sentence, you do not yet understand Floyd–Warshall.

**Three facts that come free with the matrix:**

- $d[i][i] < 0$  after the run  $\Leftrightarrow$  vertex  $i$  lies on a negative-weight cycle. The cheapest global negative-cycle detector there is, and it names the vertices involved.

- **Diameter** = `max` over finite `d[i][j]`; **centre** = `argmin_i max_j d[i][j]`.
- Swap `(min,+)` for `(max,min)` and you get **widest-path** / **bottleneck** capacities; for `(+,×)` on a DAG you get **path counts**. Same loops, different algebra.

**Why it beats  $v \times$  Dijkstra despite matching asymptotically:** three flat arrays, unit-stride inner loop, no heap, no pointer chasing, perfectly predictable branches. Constant factors, and this is the canonical example of them mattering.

## A6 JOHNSON

*Pseudocode: Part 8.*

```
// All-pairs shortest paths on a SPARSE graph that may have
// negative edges.
// Fills D[u][v]; returns false if a negative-weight cycle exists.
bool johnson(const WGraph& g, vector<vector<long long>>& D) {
    const int n = g.n;

    // 1  G' = G plus a new source s with a 0-weight edge to EVERY
    // vertex.
    // Why the new vertex: it guarantees every vertex is reachable,
    // so h is
    // finite everywhere and Bellman-Ford sees every negative
    // cycle. It cannot
    // change any delta(u,v) for u,v in V, because s has NO
    // INCOMING EDGES and
    // therefore lies on no u -> v path.
    WGraph gp(n + 1);
    for (const WEdge& e : g.edges) gp.addEdge(e.u, e.v, e.w);
    for (int v = 0; v < n; ++v) gp.addEdge(n, v, 0);

    BellmanFordOutput bf = bellmanFord(gp, n);           // 2 one
    Bellman-Ford run
    if (!bf.ok) return false;                             // 3
    negative-weight cycle

    // 4-5  h(v) = delta(s, v). Always <= 0, since the 0-edge s->v
    // is one option.
    vector<long long> h(bf.r.d.begin(), bf.r.d.begin() + n);

    // 6-7  w-hat(u,v) = w(u,v) + h(u) - h(v).
```

```

//
// WHY THIS WORKS (Lemma 23.1): summing along a path, every
interior h
// cancels and only the endpoints survive:
//      w-hat(p) = w(p) + h(v_0) - h(v_k)
// The correction depends ONLY on the endpoints, not on the
path or its
// length -- so EVERY u->v path shifts by the same amount and
the RANKING is
// preserved. Cycle weights are completely unchanged (v_0 ==
v_k).
//
// AND w-hat >= 0, by the triangle inequality:
//      delta(s,v) <= delta(s,u) + w(u,v)  <=>  h(v) <= h(u) +
w(u,v)
//
//                                     <=>  w(u,v) + h(u)
- h(v) >= 0
// One line of Chapter 22 makes Dijkstra legal on a graph with
negative edges.
//
// NOT to be confused with the WRONG fix (Exercise 23.3-4):
subtracting the
// minimum weight from every edge penalises paths with MORE
edges, so it
// changes which path is shortest.
WGraph gh(n);
for (const WEdge& e : g.edges) gh.addEdge(e.u, e.v, e.w +
h[e.u] - h[e.v]);

D.assign(n, vector<long long>(n, PATH_INF));
for (int u = 0; u < n; ++u) {                                // 9  for
each u in V
    PathResult r = dijkstra(gh, u);                            // 10
    DIJKSTRA on w-hat
    for (int v = 0; v < n; ++v)
        if (r.d[v] != PATH_INF)
            D[u][v] = r.d[v] + h[v] - h[u];                    // 12
(23.11) undo the shift
// NOTE THE SIGN FLIP: edges use +h(u) - h(v);
distances use
// +h(v) - h(u). Getting this backwards is the
classic Johnson

```

```

        // bug, and it produces plausible wrong numbers.
    }
    return true; // 13 return
D
}

```

**Complexity.**  $O(V^2 \lg V + VE)$  with Fibonacci heaps,  $O(VE \lg V)$  with binary heaps. Breakdown: one  $O(VE)$  Bellman-Ford,  $\Theta(E)$  reweighting, then  $V$  **Dijkstras, which dominate**. Space  $\Theta(V^2)$  for the output,  $\Theta(V+E)$  besides.

**When to use which:**

|                            | SPARSE $E \approx V$ | DENSE $E \approx V^2$                                             |
|----------------------------|----------------------|-------------------------------------------------------------------|
| <b>Johnson</b>             | $O(V^2 \lg V)$ —     | $O(V^3 \lg V)$                                                    |
|                            | <b>wins</b>          |                                                                   |
| <b>Floyd–<br/>Warshall</b> | $\Theta(V^3)$        | $\Theta(V^3)$ — <b>wins</b> , and by more than the $\lg$ suggests |

**Johnson is the best example in either book of composing three techniques.** The reweighting scheme is a system of **difference constraints** ( $h(v) - h(u) \leq w(u, v)$ ), Part 5); **Bellman-Ford** solves that system; **Dijkstra** exploits the result. The same "reweight so a faster algorithm becomes legal" move reappears in min-cost max-flow, where Johnson potentials are exactly what lets successive-shortest-paths run Dijkstra instead of Bellman-Ford — and in  $A^*$ , where a consistent heuristic  $h$  makes the search *literally* Dijkstra on  $w(u, v) + h(v) - h(u)$ , the same formula.

---

\*Next: [M16 — Network Flow & Matching \(planned\)](#) (CLRS 24–25 + Skiena 8.5) — max-flow min-cut, Ford-Fulkerson and Edmonds-Karp, Dinic, and matching as a flow problem.\*